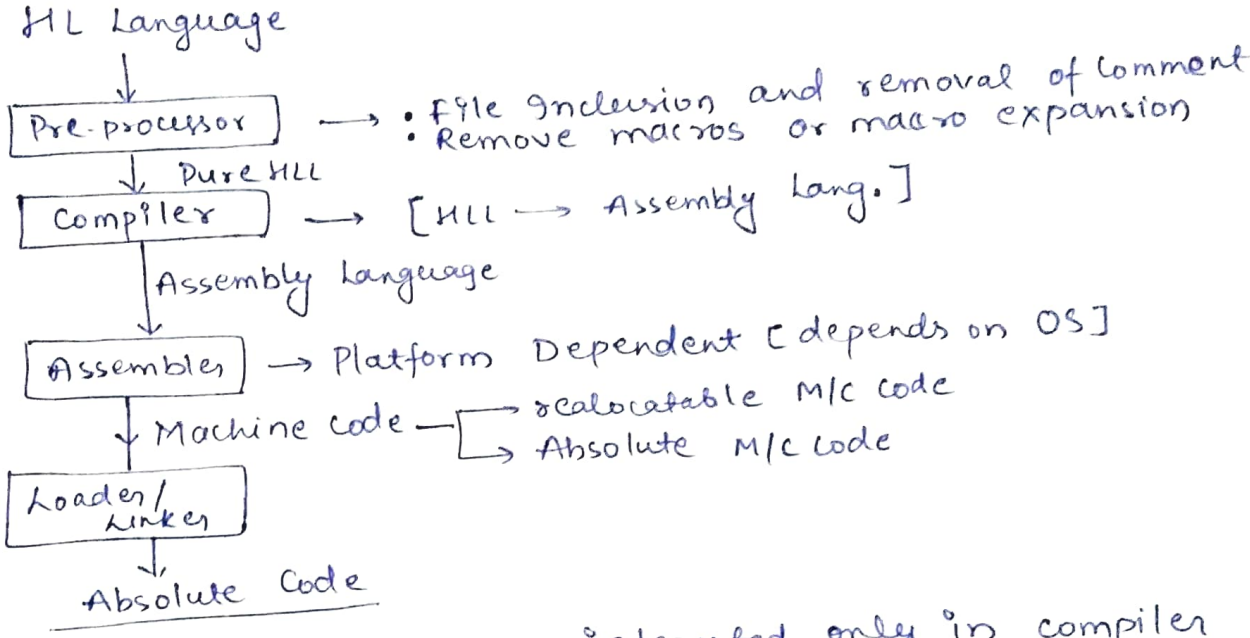
Compiler Design

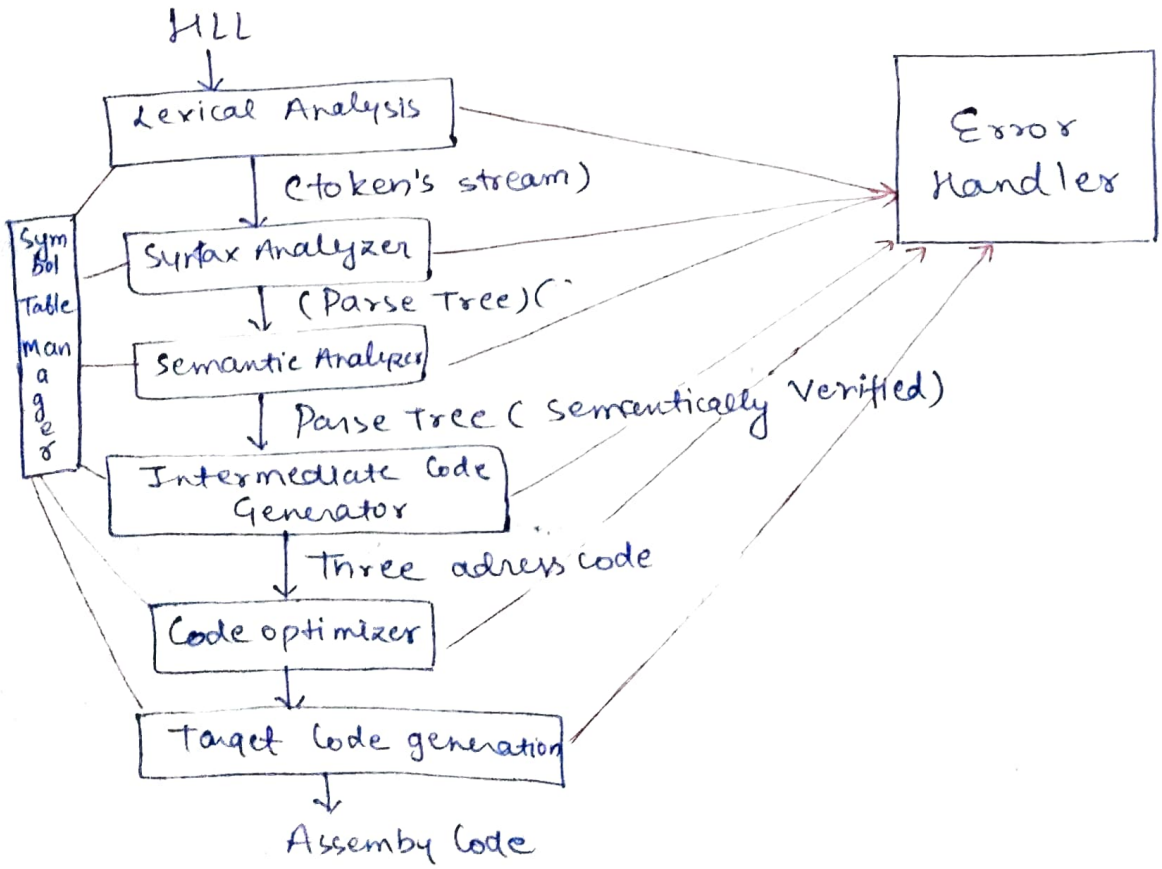
[2-5 Marks]

Introduction to various phases of Compiler

Compiler converts HL language to low-level language (0/1).



But in compiler design we are interested only in compiler phase means conversion of pure HLL to Assemby Lang.



Ex → $x = a + b * c$;

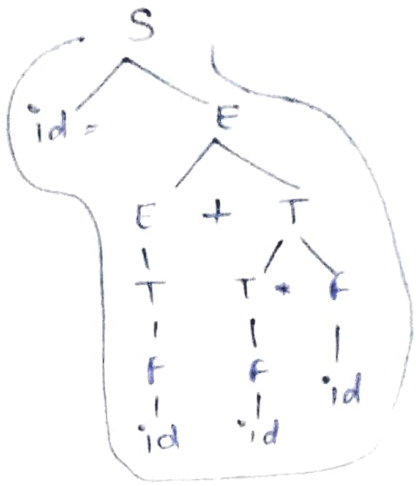
(lex) [Lexical Analyzer] → Remove white spaces

id = id + id * id → To identify a identifier we can use regular expression

(Yacc) [Syntax Analyzer] → Use set of production

Statement (S) → id = E ;
Expression (E) → E + T / T
Term (T) → T * F / F
factor (F) → id

(Parse Tree)



⇒ [Semantic Analyzer]

Parse tree semantically verified

[Intermediate code Generator]

Frontend
Backend

Three address code because it has only 3 variable required at max 3 addresses

$t_1 = b * c$;
$t_2 = a + t_1$;
$x = t_2$

[Code Optimizer]

$t_1 = b * c$;
$t_2 = a + t_1$

[Target Code Generator]

mul R1 R2
add R0, R2
mov R2, x

Example is just to explain the working → इसका मतलब हर जगह सारे statements given हों जरूरी नहीं है।

LANCE → It provides tools to make the frontend part of compiler

'lex' is used to perform lexical analysis and Yacc is used for syntax analysis

⇒ Backend is system dependent.

Symbol Table

* It is an important data structure created and maintained by compilers in order to store information about the occurence of various entities such as variable, class, function, objects and interfaces.

* The information is collected by the 'analysis phase' of a compiler and used by the 'synthesis phase' to generate target code.

Usage of symbol table by all the phases of compiler:

Phases	Usage
(1) Lexical Analysis	→ Creates new entries for each new identifier
(2) Syntax Analysis	→ Add information regarding attributes like type, scope, dimension, life of reference & line of use.
(3) Semantics Analysis	→ Use the available info. to check for semantics
(4) Intermediate Code Generator	→ Information in symbol table helps to add temp. variable information
(5) Code Optimization	→ Info. in symbol table used in machine-dependent optimization by considering address and aliased variable information.
(6) Target Code Generation	→ Generate the code by using the address info. of identifiers.

Symbol Table Entries

* Each entry in the symbol table is associated with attributes that support the compiler in different phases.

The attributes are –

① Name ② Size ③ Dimension ④ Type
⑤ Line of declaration ⑥ Line of usage
⑦ Address

→ Line of declaration contains info. of line in which variable is declared. It helps if you have an error then it will revert back the line of errors

⇒ All the attributes are not fixed size –

Limitation of fixing the size –
→ if chosen small it can't store more variables.
→ if chosen large lot of space wasted.

So, the size of symbol table should be dynamic to allow increase in size at compile time.

→ Line of usage helps in a way if a variable used in multiple lines.

Operations on the symbol table

Dependent on whether the language is block-structured or non-block structured.

Non-block structured: { }

* contains only one instance of the variable declaration and its scope is throughout the programme.
* for non-block structured languages operations are –
  ⇒ Insert
  ⇒ Look up

Block-structured: { { { } } } → Blocks

* variables may be re-declared and scope is within the block.
* Operations –
  ⇒ Insert ⇒ Lookup ⇒ Set (start a new one) ⇒ Reset (remove it completely)

Gate: 2012 Access time of a symbol table is logarithmic if it is implemented in:

① Linear List → Also called as linked list. In linked list implementation it will take $O(n)$.

② Search tree → Also consider as BST. So it will take $\Theta(\log_k n)$ time where $k \rightarrow$ no. of children

③ Hash Table → $O(1)$

④ None of the above → X

So, Ans → (B) Search Tree

Various Implementation on Symbol Table

Implementation	Insertion Time	Lookup Time	Disadvantages
① Linear List			⇒ Lookup time ∝ table size
(a) Ordered			
→ Array	$O(n)$	$O(\log n)$	⇒ Every insertion operation preceeded with lookup operation
→ Linked List	$O(n)$	$O(n)$	
(b) Unordered	$O(1)$	$O(n)$	
② Self Organizing List (Unsorted)	$O(1)$ [inserted at beginning]	$O(n)$	⇒ Poor performance when less frequent item searched.
③ Search tree	$O(\log_k n)$	$O(\log n^k)$	⇒ We always keep it balanced.
④ Hash table	$O(1)$	$O(1)$	⇒ When there are too many collisions T(n) may be $O(n)$

→ We consider no duplicate elements in all four Implementations.

Lexical Analyzer and Grammar

→ Phase that reads character by character and produce tokens also remove comments and white spaces.

→ Complete string is consider as a single token,

Ex "|%d|a+|b|" → not 6 tokens

↓

only one token.

Grammar = (V, T, P, S)

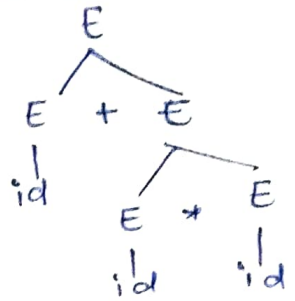
$E \longrightarrow E+E \mid E*E \mid id$

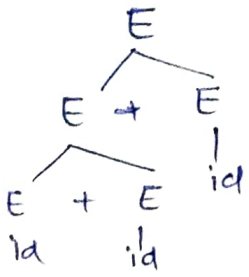
To achieve id + id * id → Left most Derivation

$E \rightarrow E+E$
$\rightarrow id+E$
$\rightarrow id+E*E$
$\rightarrow id+id+E$
$\rightarrow id+id*id$

$E \rightarrow E*E$
$\rightarrow E+E*E$
$\rightarrow id+E*E$
$\rightarrow id+id*id$

Parse tree





→ Since we are getting two parse tree so Grammar is ambiguous.

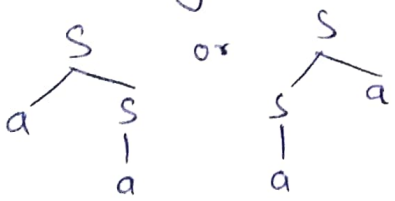
→ If a Grammar have two Right most derivation or two Left most derivation or two Parse tree then grammar is ambiguous.

There is no direct procedure for
→ To find grammar is ambiguous or not
→ To make grammar unambiguous.

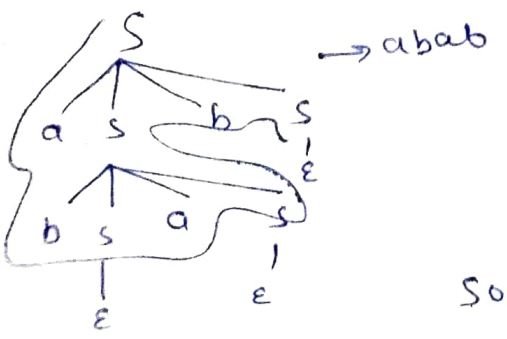
Imp → Ambiguity problem is Undecidable.

Ex ① $S \longrightarrow as \mid sa \mid a$] so ambiguous.

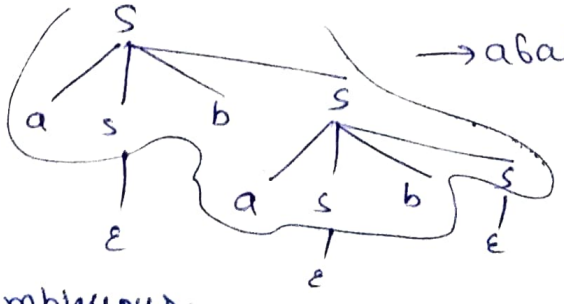
Assume a string $\omega = aa$



② $S \longrightarrow asbs \mid bsas \mid \epsilon$



→ abab



→ abab

So ambiguous.

There is no algorithm so need to it by making parse trees.

Errors and their recovery in lexical analysis

Errors: ① Numeric Literals are too long (Ex→ int A = 12345876)
② Long identifiers → longer than length given in regular expression.
③ Ill-formed numeric literals (int A = $1234)
④ Input characters are not in language.

Error Recovery:

1) **Delete:** Unknown characters are deleted (Panic mode recovery)
Ex: "Chars" corrected as "char" by deleting 's'.

2) **Insert:** Missing character is inserted to form a meaningful tokens.
Ex: "cha" corrected as "char" by inserting 'r'.

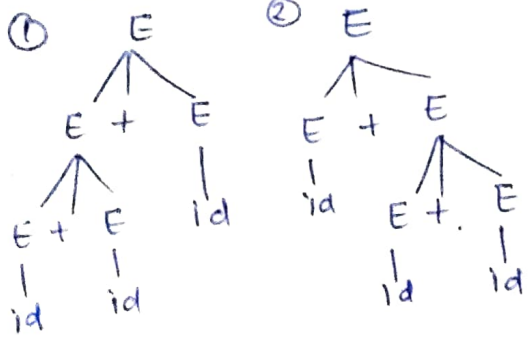
3) **Transpose:** Based on certain rules we can transpose two characters.
Ex: "whejl" can be corrected to "while"

4) **Replace:** Based on replacing one character by another.
Ex: "chrr" can be corrected as "char".

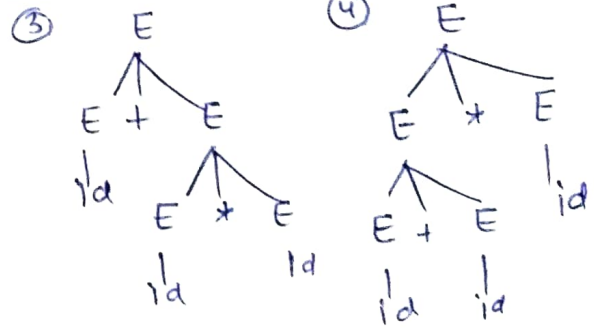
Ambiguous Grammars and making them unambiguous

Ex- $E \rightarrow E+E \mid E*E \mid E$

id + id + id



id + id * id



→ Ambiguity generates if a ~~operand~~ operator have operators on its both sides

→ In ① and ② '+' is a operator which is following left associativity rules so, ① is true and ② is false but there are two trees so grammar is ambiguous and problem is associativity.

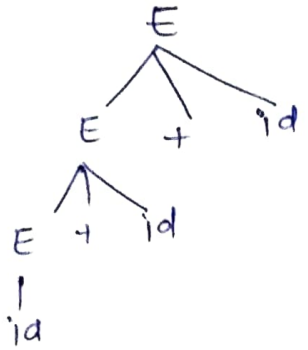
In (3) and (4) we have two operators of different precedence '+' and '*' and '*' have more precedence so '*' have to be evaluated first. But in (4) '+' evaluated first. So grammar is ambiguous with precedence issue.

In (1) and (2) we want growth in left side of tree so we need to stop the growth in right side. So grammar must be left recursive.

(Left most Symol) RHS = LHS.

$E \rightarrow E+id \mid id$

id+id+id



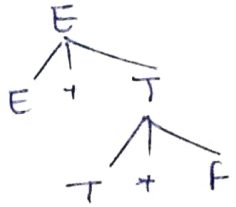
In (3) and (4) we found that highest precedence should be at lower most level.

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow id$

id+id+id



2 marks

Note: To maintain associativity use recursion(left).
To maintain precedence use levels.
To maintain power use right recursion.

Note: for power operator as ↑

Ex 2↑3↑2 → $((2)^3)^2$

Ex: $E \rightarrow E+T \mid T$ (Left recursive) → Generate first operate last

$T \rightarrow T*F \mid F$ → ——— second ——— second

$F \rightarrow G \uparrow F \mid G$ → ——— last ——— first

$G \rightarrow id$ → Right Recursive

↑ Precedence

Ex → (1) bEx → bEx or bEx | bEx and bEx | not bEx | True | false

bEx → boolean expression

E → E or F | F
F → F and G | G
G → NOT G | True | false

Unambiguous Grammar (Left Recursion)

(2) R → R+R | RR | R* | a | b | c

E → E+T | T
T → T*F | F
F → F* | a | b | c

Unambiguous

R → Regular Expression

(3) A → A $ B | B
B → B # C | C
C → C @ D | D
D → d

Precedence order $ < # < @

A → B → C → Left recursive Production so Left associativity.

(4) E → E*F | F+E | F
F → F - F | id

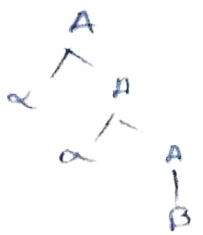
Grammar is ambiguous bcoz -
① '*' is left recursive and '+' is right recursive here
② '*' and '+' both given as same precedence.
③ '-' given more precedence over '*'.

Elimination of left recursion and left factoring the Grammars:

Recursion

Left Recursion
A → Aα | β
Language ← [βα*]

Right Recursion
A → αA | β
[α*β] → language



TOP-DOWN Parsers doesn't operate on Left-Recursion.
So we try to remove Left Recursive Grammars.

Given $A \rightarrow \beta\alpha^+$ but we want to remove α^+

So, Soln

$$A \rightarrow \beta A'$$
$$A' \rightarrow \epsilon \mid \alpha A'$$

→ Right recursive Grammar equivalent to Left recursive Grammar

(2) $E \rightarrow E+T \mid T$ ($A \rightarrow A\alpha + \beta$ form)

A : E ; α : $+T$; β : T

$(A \rightarrow \beta A')$: $E \rightarrow TE'$

$E' \rightarrow \epsilon \mid +TE'$

(3) $S \rightarrow S0S1S \mid 01$

A : S ; α : $0S1S$; β : 01

Soln $A \rightarrow \beta A'$: $S \rightarrow 01S'$

$S' \rightarrow \epsilon \mid 0S1SS'$

(4) $S \rightarrow L \mid x$

$L \rightarrow L,S \mid S$

A : L ; α : $,S$; β : S

$$A \rightarrow SA'$$
$$A' \rightarrow \epsilon \mid ,SA'$$
$$A' \rightarrow L$$

$\approx$

$$L \rightarrow SL'$$
$$L' \rightarrow \epsilon \mid ,SL'$$

(5) $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \cdots \mid \beta_1 \mid \beta_2 \mid \cdots$

Soln

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \beta_3 A' \mid \cdots$$
$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \cdots \mid \epsilon$$

Grammars type

(1) Ambiguous and Unambiguous

(2) Left Recursive and Right Recursive.

We don't want ambiguous and Left Recursive grammar.

③ Deterministic and non-deterministic Grammar

$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$ (Multiple option)

Problem → Backtracking not Allowed

Common prefix problem → Since each element ($\alpha\beta_1, \alpha\beta_2, \alpha\beta_3$) have '$\alpha$' as prefix. So it may create ambiguity as-

Suppose input is $\alpha\beta_3$ then, parser have to decide which have to choose from ($\alpha\beta_1, \alpha\beta_2$ and $\alpha\beta_3$) just by looking α. Suppose it choose $\alpha\beta_2$ and we have $\alpha\beta_3$ in input then it needs to backtrack and it is not possible.

Soln

$A \to \alpha A'$

$A' \to \beta_1 \mid \beta_2 \mid \beta_3$] Left Prefixing or factoring. (Deterministic)

→ Conversion of Non-deterministic to Deterministic called 'left factoring'.

Ex ① $S \to iEtS \mid iEtSeS \mid a$

$E \to b$

Soln

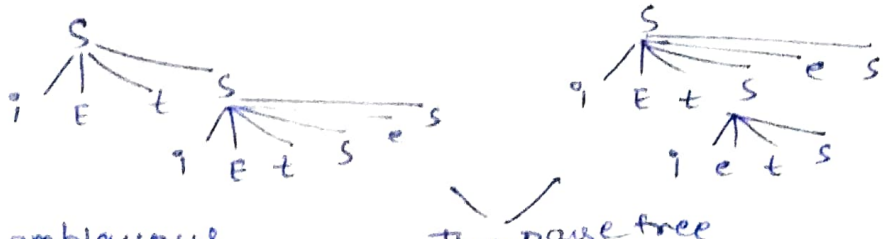
$S \to iEtSS' \mid a$

$S' \to \epsilon \mid eS$

$E \to b$

Check for ambiguity

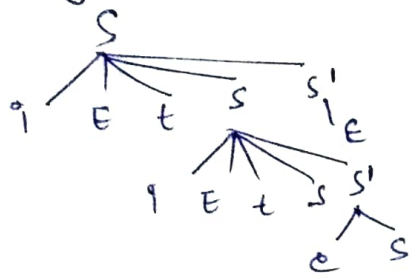
Take a string: iEtiEtSeS (Intermediate string of parse tree)



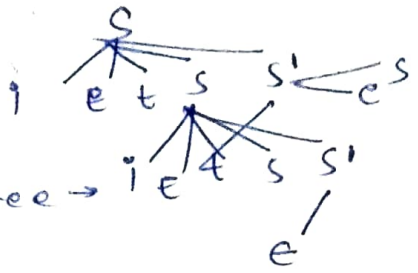
two parse tree

→ So, grammar is ambiguous before Left factoring

After left factoring



← 2 Parse tree →



→ Eliminating non-determinism doesn't guaranteed ambiguity removing.

→ If we have if statement more than else statement then it cause a problem called dangling-else problem.

Soln → Always associate else with its nearest if.

Remove Non determinism

Ex- ① $S \rightarrow assbs \mid asasb \mid abb \mid b$

either pull 'a' as common prefix from all '3'

Pull 'as' as prefix from start 2

$S \rightarrow ass' \mid abb \mid b$

$S' \rightarrow sbs \mid asb$

① $S \rightarrow as' \mid b$

$S' \rightarrow ssbs \mid sasb \mid bb$

③ steps in soln

② $S' \rightarrow ss''$

③ $S'' \rightarrow sbs \mid asb$

Ex ② $S \rightarrow bssaas \mid bssasb \mid bsb \mid a$

Soln $S \rightarrow bss' \mid a$

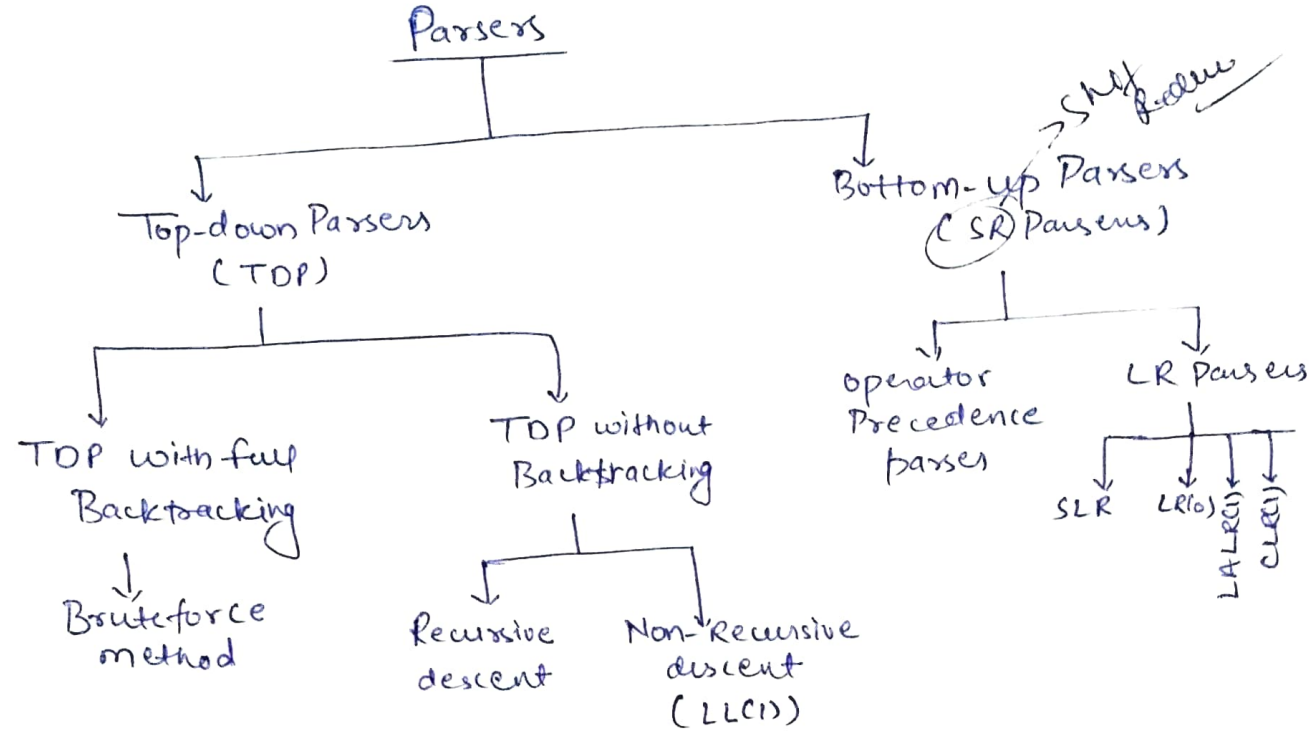
$S' \rightarrow saas \mid sasb \mid b$: $S' \rightarrow sas'' \mid b$

$S'' \rightarrow as \mid sb$

Reduces to

Parsers (Syntax Analyzer)

Introduction to Parsers, LL(1) Parsing

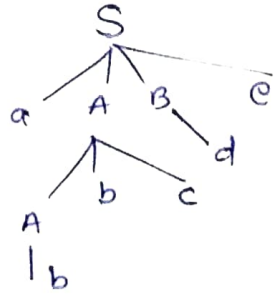


→ TDP without backtracking can be done by removing left recursion (To accept top-down necessary) and non-determinism. (for non backtracking)

→ 'operator precedence parser' is only parser that can accept ambiguous grammar.'

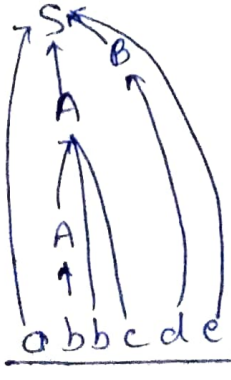
Parsers task is to verify that given string can be generated or not using parse tree.

Ex → S → aABe
A → Abc | b
B → d



Top to down selection of production.
what to use?

String(w) = abbcde



→ when to reduce?
→ what to use?

→ Bottom-to-up approach

Top down parser

$S \Rightarrow aABe$
$\Rightarrow aAbcBe$
$\Rightarrow abbcBe$
$\Rightarrow abbcde$

→ Way to acheive the paused string is Left most derivation

Bottom up parser

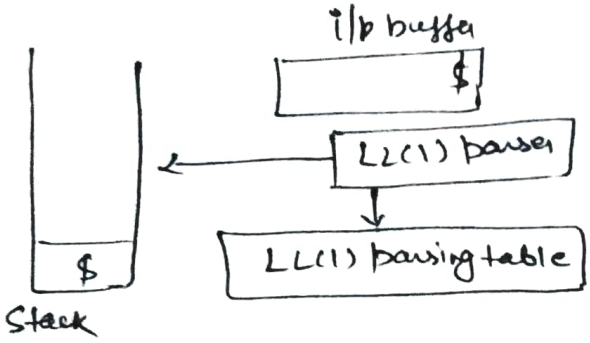
$S \Rightarrow aABc$
$\Rightarrow aAde$
$\Rightarrow aAbcde$
$\Rightarrow abbcde$

→ Way to acheive bottom-up parsing is right most derivation in reverse.

LL(1) → no of look aheads
- from Left to right
- Left most derivation

Components of Parser

1. LL(1) parser → Contain parsing algorithm
2. LL(1) parsing table → Construct using Grammar
3. Stack → used for parsing.
4. i/p buffer → inputs with $ at bottom of stack



① First()

$S \rightarrow aABC$
$A \rightarrow b$
$B \rightarrow c$
$C \rightarrow d$

First(S) → a
(A) → b
(B) → c
(C) → a

first element in string.

Ex: $S \rightarrow ABC$
$A \rightarrow b \mid \epsilon$
$B \rightarrow c$
$C \rightarrow d$

Here first(S) will be derived from A if A derived 'b' then first(S) is b otherwise first(S) will be the value derived by the B.

② Follow()

$S \rightarrow ABCD$
$A \rightarrow b \mid \epsilon$
$B \rightarrow c$
$D \rightarrow e$
$C \rightarrow d$

follow(S) → look RHS of each production if anyone have then do the operation performed for follow(A) otherwise

follow(S) → { $ } → Dollar only

$ follows each starting symbol.

⇒ follow(A) : S → A B C D contains A so follow(A) will be the first terminal of complete RHS.

$\frac{A}{LHS}$ $\frac{B\,C\,D}{RHS}$ → first variable is B that always generated ε

⇒ So follow(A) → {c}

⇒ Similarly follow(B) → $\frac{B}{LHS}$ $\frac{C\,D}{RHS}$ = {d}

⇒ follow(C) → $\frac{C}{}$ $\frac{D}{e}$: {e}

⇒ follow(D) → if it is not follow by any variable then include $.

follow(D) = {$}

Short Note

'ε' can't be included in follow() sets but first() can contain 'ε'. It is not necessary that '$' must come in each follow().

Ex. on first and follow in LL(1):

		first()	follow()	
①	S → ABCDE	{a, b, c}	{$}	→ Grammar must not be left recursive.
	A → a/ε	{a, ε}	{b, c}	
	B → b/ε	{b, ε}	{c}	
	C → c	{c}	{d, e, $}	
	D → d/ε	{a, ε}	{e, $}	
	E → e/ε	{e, ε}	{$}	
②	S → Bb/Cd	{a, b, c, d}	{$}	
	B → aB/ε	{a, ε}	{b}	
	C → cC/ε	{c, ε}	{d}	

		first()	follow()
③	$E \to TE'$	{id, (}	{$,)}
	$E' \to +TE' \mid \epsilon$	{+, ε}	{$,)}
	$T \to FT'$	{id, (}	{+, $,)}
	$T' \to *FT' \mid \epsilon$	{*, ε}	{+, $,)}
	$F \to id \mid (E)$	{id, (}	{*, +, $,)}
④	$S \to ACB \mid CbB \mid Ba$	{d, g, h, ε, b, a}	{$}
	$A \to da \mid BC$	{d, g, h, ε}	{h, g, $}
	$B \to g \mid \epsilon$	{g, ε}	{$, a, h, g}
	$C \to h \mid \epsilon$	{h, ε}	{g, $, h, b}
⑤	$S \to aBDh$	{a}	{$}
	$B \to cC$	{c}	{g, f, h}
	$C \to bC \mid \epsilon$	{b, ε}	{g, f, h}
	$D \to EF$	{g, f, ε}	{h}
	$E \to g \mid \epsilon$	{g, ε}	{f, h}
	$F \to f \mid \epsilon$	{f, ε}	{h}

if follow(X) = ε then follow(LHS) = follow(RHS)

Construction of LL(1) parsing table

We are constructing the parsing table of set of productions ③ above

	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to id$			$F \to (E)$		

(Terminals) →

(Variable) ↑

If RHS of production contain ε then place the production in the follow() of LHS.

⊗ id

E | $E \to TE'$ means we can achieve id from E using $E \to TE'$, and place in first(T).

Ex-(2) $S \rightarrow (S) \mid \epsilon$

first(S) → { (, ϵ }

follow(S) → {), $ }

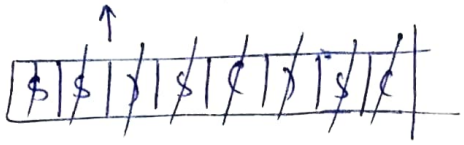
	(	)	$
S	S→(S)	S→ϵ	S→ϵ

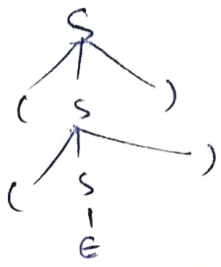
for first(S) — for ϵ

Grammar used for generating balanced parantheses

(S) → ((S)) — — (((--)))

Ex → ω = (( )) $





Top-down parser will not work if a single column in a table contains multiple productions, for each row.

Disadvantage of LL(1)

(3) $S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

	a	b	$
S	S→AaAb	S→BbBa	
A	A→ϵ	A→ϵ	
B	B→ϵ	B→ϵ	

gmb

for LL(1) parsing grammar must not be left recursive or non-deterministic.

(4) $A \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid - - - \alpha_n$

	a	b	c	d	. . .
A					

This gramar is deterministic and also not left recursive but not the suitable for LL(1) parsing because containing more than one value in same colum.

→ it can be accepted if they are mutually exclusive.

(i) $S \rightarrow asbS \mid bSaS \mid \epsilon$

	a	b	$
S	S→asbS S→ϵ (X)	S→bSaS S→ϵ (X)	S→ϵ

not possible and it is ambiguous.

(ii) $S \rightarrow aABb$

$A \rightarrow c \mid \epsilon$

$B \rightarrow d \mid \epsilon$

Problem may arises if there is an alternative as -

A → can derive c or ϵ both.

→ No ambiguous grammar is LL(1).

$S \rightarrow aB/\epsilon$
$B \rightarrow bC/\epsilon$
$C \rightarrow cS/\epsilon$

	a	b	c	$
S	S→aB			S→~~εB~~
B		B→bC		B→ε
C			C→cS	C→ε

↳ Grammar is LL(1).

Recursive descent parser (RDP)

$E \rightarrow iE'$
$E' \rightarrow +iE'/\epsilon$

```
E(){
if(l=='i')
  {
  match('i');
} } E'();
```

```
E'(){
 if(l=='+')
{ match(+)
  match('i')
} E'();
else return
}
```

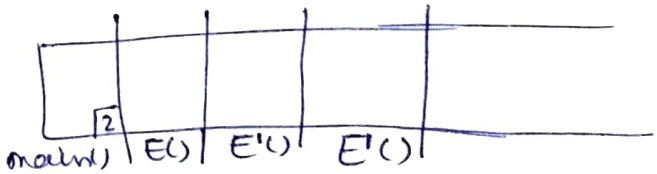
] for ε

```
match(char t)
{ if(l=='t')
   l=getchar(); ] → to increment 'l'
  else
   printf("error")
]
```

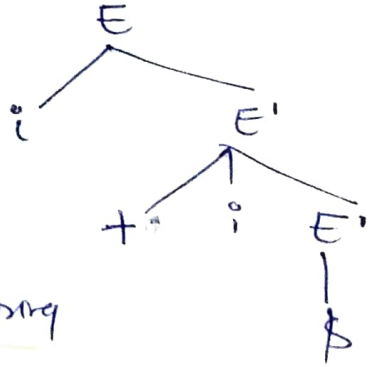
l → look ahead pointer variable

```
main(){
1.   E();
2.  if(l=='$')
3.   print("successful")
  }
```

Ex i+i$ to generate



↓
Recursion Stack



→ In RDP we make recursive functions for every variable (LHS) and then try to parse the string (Input) using functions.

→ Normal stack used for recursion explained in activation record concept

Operator grammar and operator precedence parser
(Bottom-UP Parser)

Operator Grammar

$E \rightarrow E+E \mid E*E \mid id$

In operator grammar no two operands are adjacent means operator used as separator without 'ϵ' production.

$E \rightarrow EAE$
$A \rightarrow \epsilon \mid + \mid *$
] not an operator grammar becaus EA two variables are adjacent.

$S \rightarrow SbSbS \mid SbS \mid a$] → Operator Grammar

Imp

Operator precedence parser can even parsed the ambiguous grammars

Operator Relation Table

$E \rightarrow E+E \mid E*E \mid id$ → ambiguous.

	id	+	*	$
id	x	>	>	>
+	<	>	<	>
*	<	>	>	>
$	<	<	<	x

(Terminals)

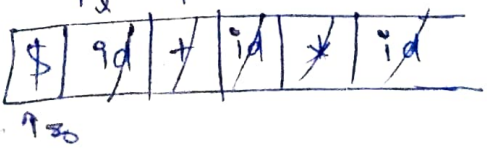
* Two id together means two variables together which is not possible (x).
* id have highest precedence over all operators
* $ have least precedence

⇒ '+', '*' have left associativity. So ~~left~~ in comparison of two same operators decision should be taken on the basis of precedence / associativity.

Left(+) > Right(+).

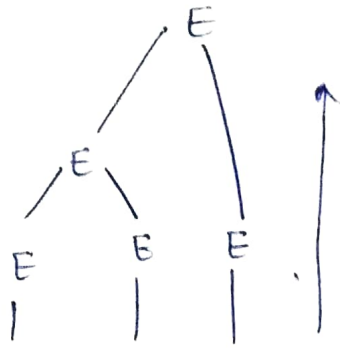
Ex → id + id * id $

E E E



if ($i > z_0$) push (i) else pop (z_0)

Reason → Pop(id) mean reduce it to variable as LHS as we doing in Bottom up Parser.

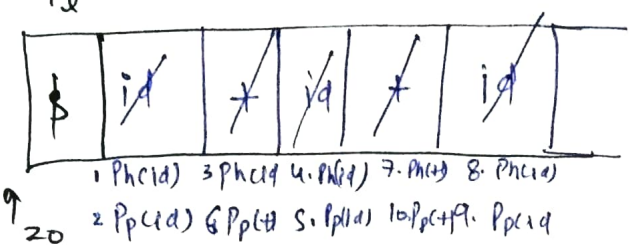


Bottom up →

Even the grammar is ambiguous but we get just a single parse tree due to precedence rule defined in table.

② id + id * id $

Ph → Push( )
Pp → Pop( )

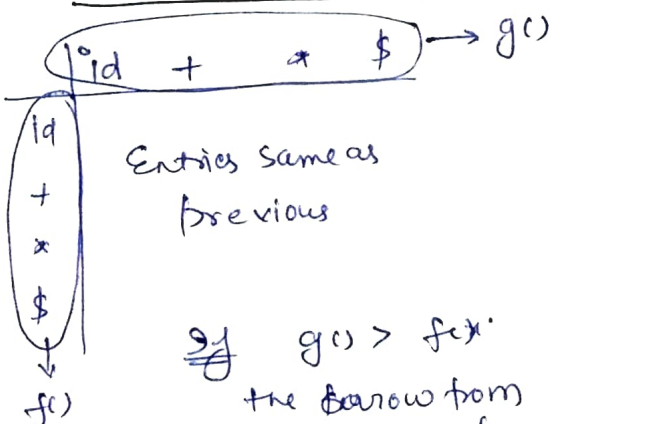


Note → open brace '(' and close brace ')' will get same (=) precedence

Disadvantage of table: size → $O(n^2)$ ∀ n → no. of operator

To reduce the size we go for,

Operator function Table

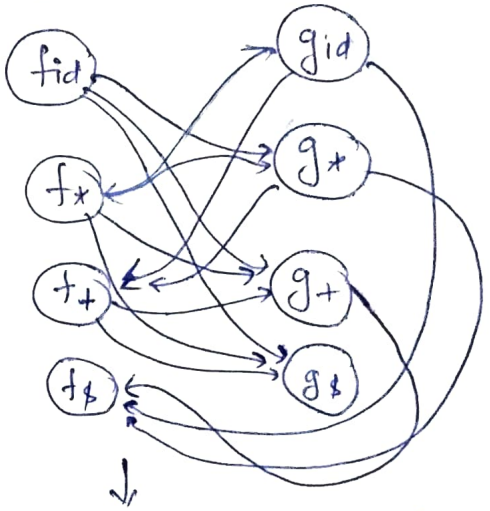


Entries same as previous

If g( ) > f*
the arrow from g → f

Ex gid > f+
So → from gid → f*

function nodes



Graph has no cycle
If there is any cycle it means you can't construct the function table

Longest path from (i) fid

fid → g* → f+ → g+ → f$

(ii) **from gid**

gid → f* → g* → f+ → g+ → f$

After finding longest path from each node than Calculate no. of node in the path and insert (n-1)

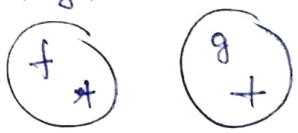
→ To see

	id	+	*	$
f	4	2	4	0
g	5	1	3	0

function table

O(2n)

f node are left side and g node are right side



4 > 1 → * > +

This way you can take decisions.

Disadvange of function table :

Blank entries in operator relation table is converted to entries in function table

error

→ Error detecting capability of function table is less than relation table.

→ But we generally use function table.

Ex

P → SR | S
R → bSR | bS
S → wbS | w
w → L*w | L
L → id.

not an operator Grammar

Put R in P

P → SbSR | SbS | S

P → SbP | SbS | S

× R → not recursive now

S → same as previous.

w →

L →

Operator reln table

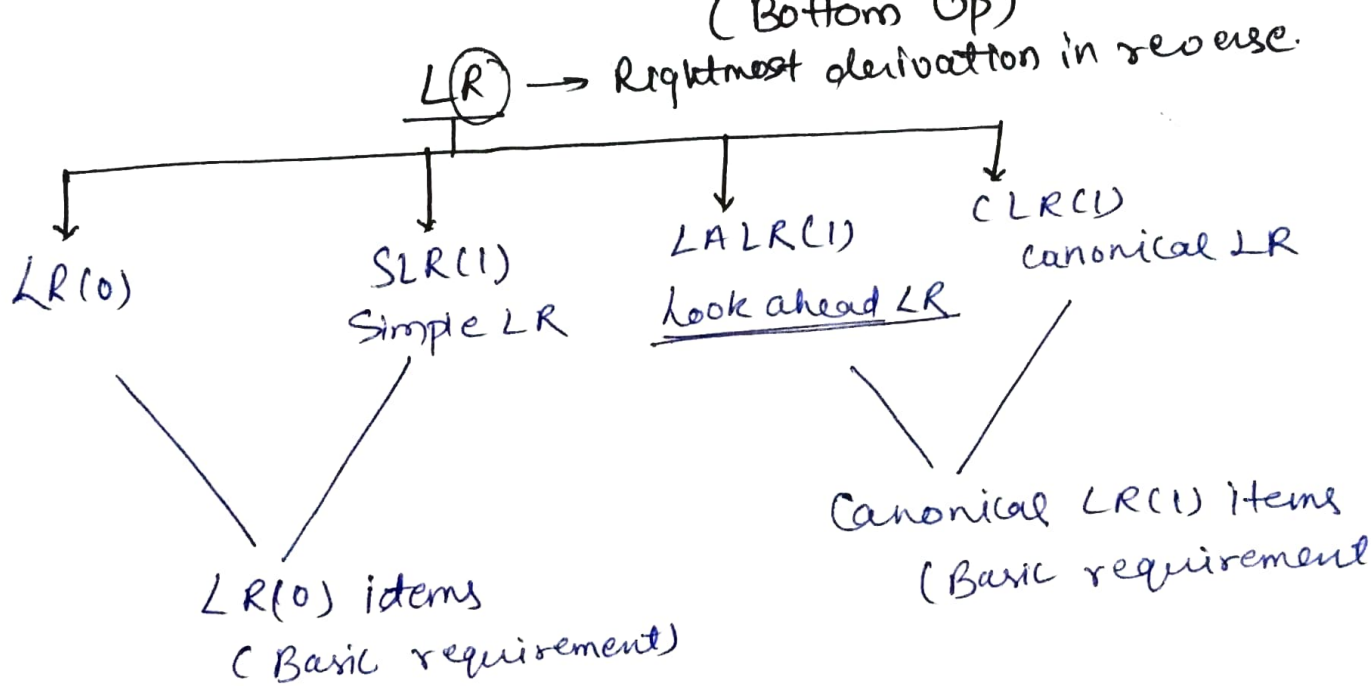
	id	*	b	$
id	×	>	>	>
*	<	<	>	>
b	<	<	<	>
$	<	<	<	×

⇒ L*w is right recursive so star(*) at right will get higher precedence over left (*).

⇒ Precedence can be decided according to level of occuring of symbol.

LR Parsing, LR(0) items and LR(0) parsing table

(Bottom Up)



Ex $S \rightarrow AA$

$A \rightarrow aA \mid b$

↓ $S' \rightarrow S$

Augmented Grammer

LR(0) Parsing table

- closure
- goto

1	$S' \rightarrow S$
2	$S \rightarrow AA$
3	$A \rightarrow aA \mid b$

→ Just add one production to make it augmented.

Any production with dot (•) at beggining called as LR(0) item.

$S \rightarrow \bullet AA$

Position of dot can be changed as -

$S \longrightarrow .AA$ → Not seen anything on RHS [LR(0) item]

$S \longrightarrow A.A$ → seen A on RHS

$S \longrightarrow AA.$ → seen everything from RHS. [final item]

Steps

① $S' \longrightarrow .S$: As you see '.' in the beginning of product then include all production derived from variable on RHS of '.' and their respective production

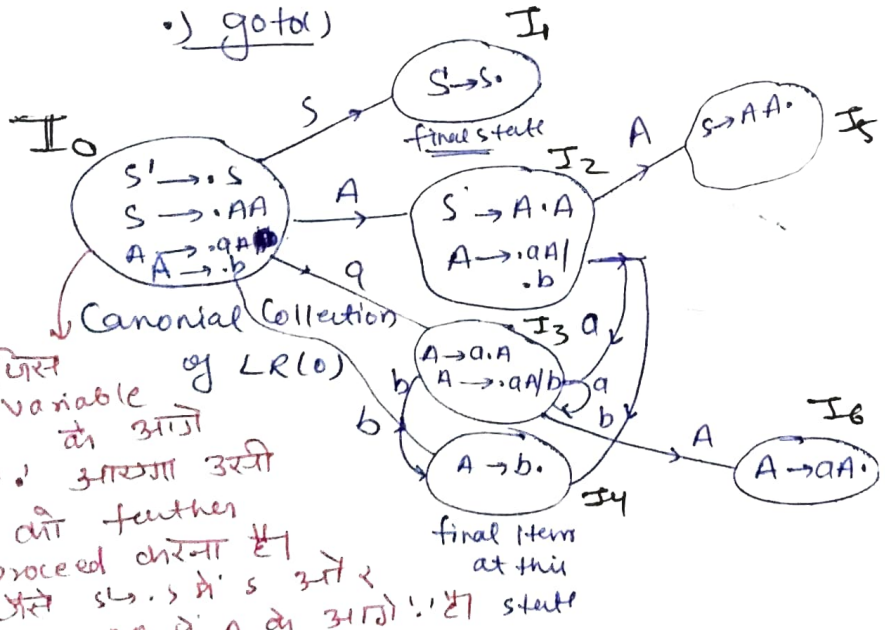
•) Closure of item: $S' \longrightarrow .S$ is

$S' \longrightarrow .S$

$S \longrightarrow .AA$

$A \longrightarrow .aA \mid .b$] Respective prodn of A.

•) goto()



Canonical Collection of LR(0)

जिस variable के आगे '.' आएगा उसी को further proceed करना है। जैसे $S \longrightarrow .S$ में S के आगे '.' है $S \longrightarrow .AA$ में A के आगे '.' है

Move '.' to one step RHS and find closure

$I_0 \longrightarrow I_6$ (7 States)

→ Same level पर indexing करो फिर next level पर आओ-वहाँ

LR(0) Parsing Table

I	(a) Action		(b) ($)	Goto(A)	(S)
0	S_3	S_4		2	1
1			accepting		
2	S_3	S_4		5	
3	S_3	S_4		6	
4	r_3	r_3	r_3		
5	r_1	r_1	r_1		
6	r_2	r_2	r_2		

→ We accept the I_1 because it derives from augmented production specially added to it. So, it needs to accept

Points to Remember

•) Assume

$S \rightarrow AA. \rightarrow r_1$
$A \rightarrow aA. \rightarrow r_2$
$A \rightarrow b. \rightarrow r_3$

] reduced move

$S_3 \rightarrow$ Shift on 3: we will do shift operation on looking a terminal from any set of production

$S_3 \rightarrow$ means I_0 on a shifts to I_3

we use numerics in case of showing transitions for variables. as.

Goto(A): 2 means I_0 on 'A' moves to I_2.

This way we use different notations for both terminal and variables.

Since I_4, I_5, I_6 are final reduced states so we write the productions whose finally reduced form represent by the state.

Ex, $A \rightarrow b$ is r_3 its reduced $\overset{\text{move}}{\Rightarrow}$ is $A \rightarrow b.$ and it is accepted by I_4.

So 4 number entry contain r_3 in all columns.

This way a complete table is designed.

LR(0) parsing example and SLR(1) table

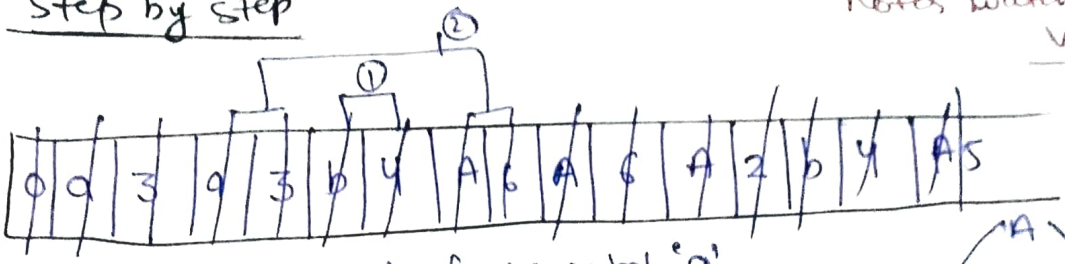
Example: $S' \rightarrow S$
$S \rightarrow AA$
$A \rightarrow aA \mid b$

] LR(0) Parsing table for given grammar prepared on previous page (backside)

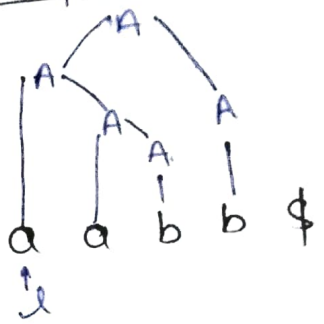
string $w =$ aabb $

→ If don't understand by notes watch the video

Step by step



Start from state 0 and first symbol 'a'



① $0 \xrightarrow{a} 3$ (Increment pointer)

② $3 \xrightarrow{a} 3$ (Increment i)

③ $3 \xrightarrow{b} 4$ (Increment i)

④ $4 \xrightarrow{b} r_3$ ($A \to b$)

Reduce $b \to A$ and then Push A in the stack. We are reducing the previous b (before 4) not current pointing b.

If in reduced production (r) have n elements on RHS then pop (2n) elements from stack.

No. of element on RHS means Ex→

$A \to a \to$ 1 element on RHS (a)

$A \to aB \to$ 2 element on RHS.

So reduced symbol $b \to q$ and pop 2 symbols from stack.

⑤ $3 \xrightarrow{A} 6$ (Pointer on b)

⑥ $6 \xrightarrow{b} r_2$ ($A \to aA$) reduce and pop 4 elements.

⑦ $3 \xrightarrow{A} 6$ (Pointer on b) ~~reduce~~

⑧ $6 \xrightarrow{b} r_2$ same as above and poped 4 elements

⑨ $0 \xrightarrow{A} 2$

⑩ $2 \xrightarrow{b} s_4$

⑪ $4 \xrightarrow{\$} r_3$ reduce $A \to b$

⑫ $5 \mid \xrightarrow{\$} r_1$ $S \to AA$ (pop 4 symbols and reduce to AA

SLR (1) → More powerful than LR(0). It also take LR(0) items to parsing.

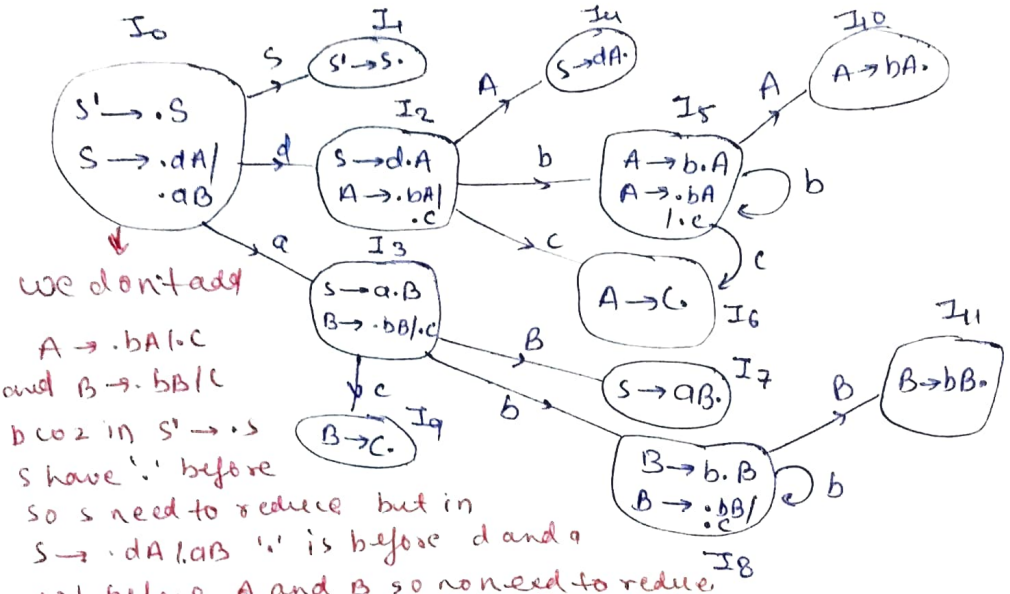
It avoids blind reduction. (Remaining part after next Qus) +11 ₹9

Ex- S → dA | aB
A → bA | c
B → bB | c

To find that Grammar is
i) LL(1) or not
ii) LR(0) or not
iii) SLR(1) or not

Do this question after reading this

→ There is no 2 production which comes in same row/column on finding first(variable). So it is definitely LL(1).



we don't add
A → .bA/.C
and B → .bB/C
bcoz in S' → .S
S have '.' before
so S need to reduce but in
S → .dA/.aB '.' is before d and a
not before A and B so no need to reduce

Canonical Collection of LR(0) items

If grammar follows LR(0) then it must be SLR(1) because SLR(1) is reduced form of LR(0).

The grammar is LR(0) because no final state is having any reduce move or shift move (RR / SR conflict).

final state means '.' at the end.

Remaining part of previous question

SLR(1) says whenever there is a final state then don't put the reduce moves blindly.

SLR(1)	action			goto	
	a	b	$	A	S
4	r_3	r_3	r_3		
5			r_1		
6	r_2	r_2	r_2		

A
|
a a b b $
↑
SLR(1) says that — cont.

On reducing the variable 'b' to 'A' verify that the variable or terminal just after 'b' (here 'B') must be in follow of A otherwise don't reduce.

So follow(A) = { $, b, ... } → since it contains 'b' so we can reduce $b \rightarrow A$.

Remember ↓

It can be said as $[A \rightarrow b.,\ r_1]$ can be placed if the variable next to 'b' (here 'b') is in follow of LHS of 'r_1'.

SLR(1) detects the error earlier than LR(0).

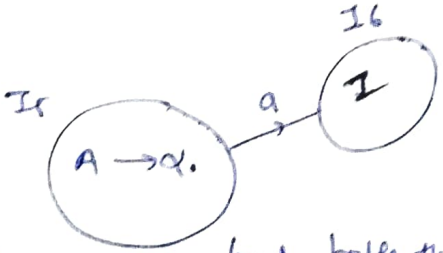
How to handle the error?

→ Provide pointer to function on blank space. so that function can print the error messages.

A Grammar can't be in LR(0) due to reasons –

① SR Conflict

Ex:

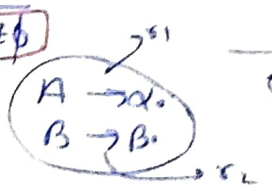


	a	b
5	$S6/r_1$	r_1

In I5 we need to put both the things the reduce move and the shift move.

② RR conflict

follow(A) ∩ follow(B) ≠ ϕ



	a	b	c
5	r_1/r_2	r_1/r_2	r_1/r_2

It generate the conflicts btw two reduce moves

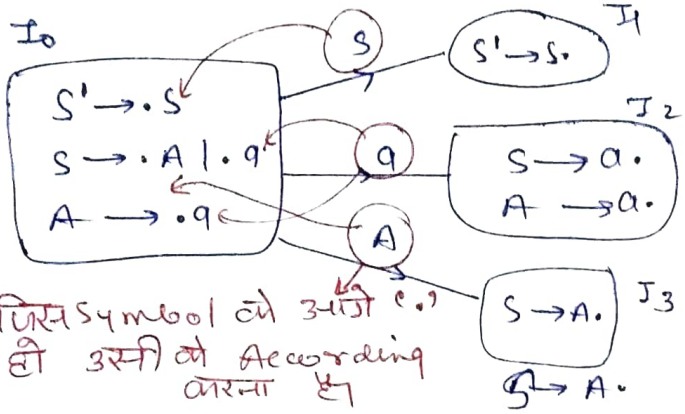
If in canonical LR(0) items graph there is no SR / RR conflicts then it must be in LR(0).

Drop

If grammar is LR(0) then it must be SLR(1), but if grammar is not LR(0) it doesn't mean that it will not be SLR(1).

Ex → S → A|a
A → a

① Not suitable for LL(1) bcoz first(S) = {a, a} and it is also ambiguous. Short Notes



→ Two reduce move in same state so not a LR(0).

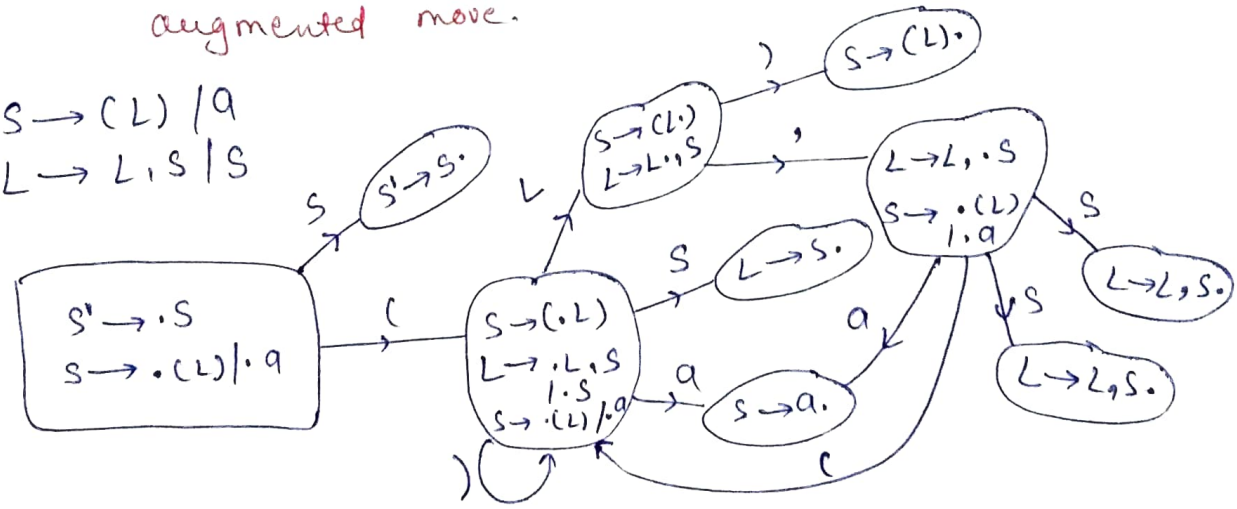
→ जिस Symbol को आगे dot हो उसी को According करना है

for SLR(1) find follow(S) = $ and follow of(A) is $.
so two entries needs to placed in same column not possible.

→ Note → ① Left recursive grammar can't be LL(1).

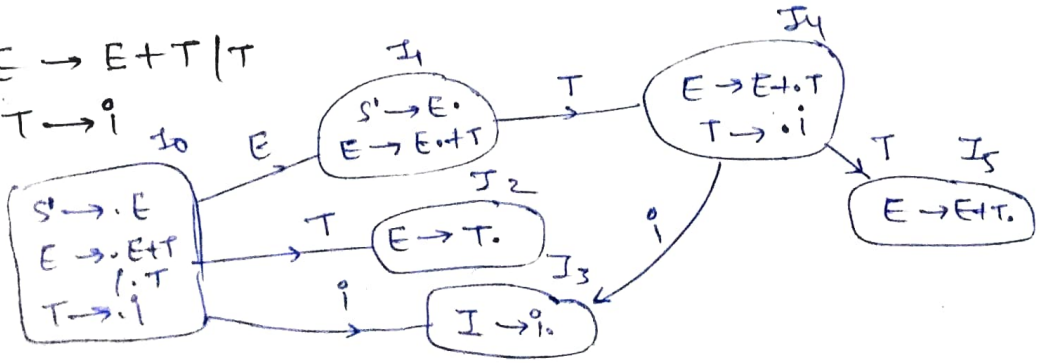
② for LR(0) & LR(1) check only final states except augmented move.

S → (L) | a
L → L,S | S



→ Grammar is not LR(0).
→ Grammar follows LR(0) and SLR(1).

E → E+T | T
T → i



We can assume that I_4 is a final state due to $S' \to E.$ and its also have reduce move $E \to E.+T$ and shift move on T. So there is SR conflict.

But we don't consider $S' \to E.$ as a final state bcoz it is augmented production which is accepting, but not a final state. So there is no SR conflict.

Ex

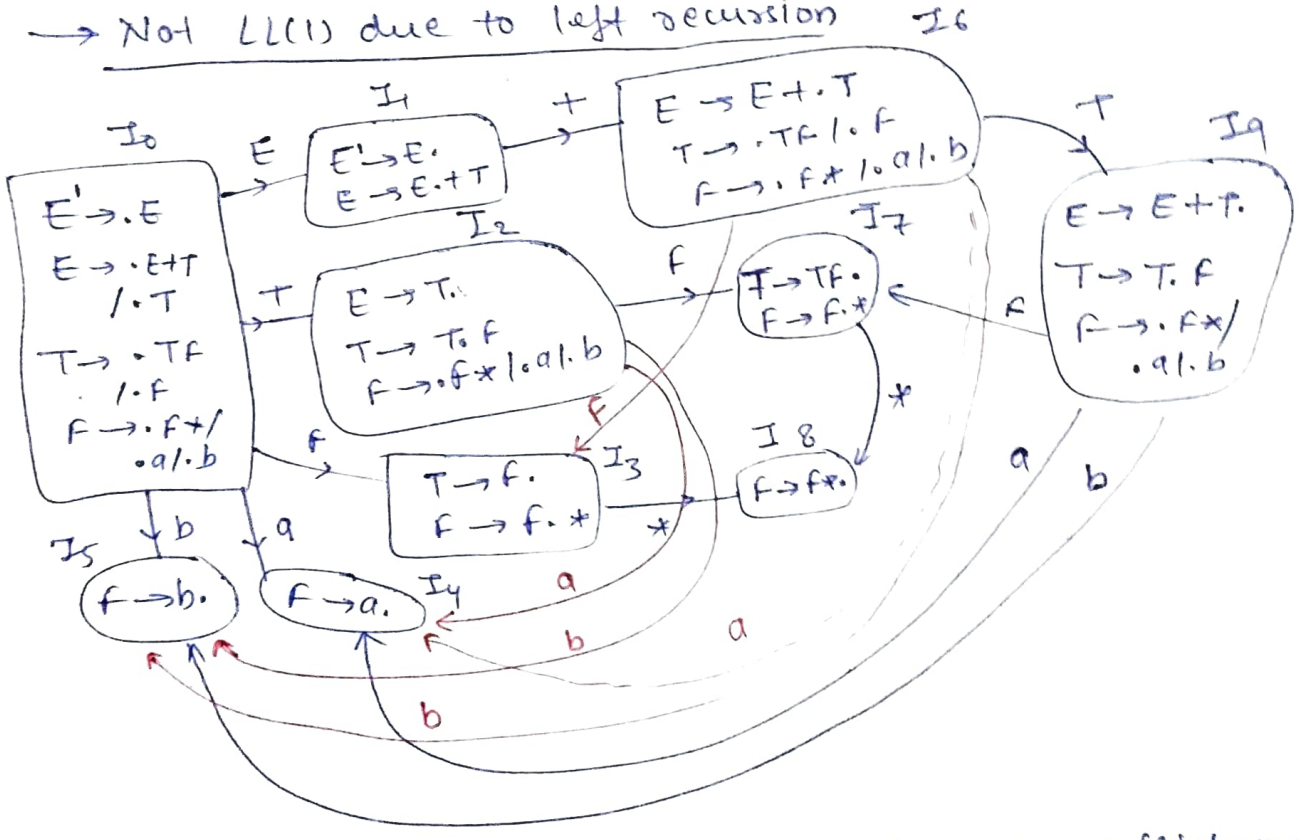
$E \to E+T \mid T$ (1, 2) → Union

$T \to TF \mid F$ (3, 4) → Concatenation

$F \to F* \mid a \mid b$ (5, 6, 7) → simple premitive / Kleen closure

this grammar represents regular expression

→ Not LL(1) due to left recursion



→ Grammar is not LR(0) due to SR conflict on I_2.

→ If needed then generate parsing table for only those reductions generated the conflicts, or conflicts generally observed in action part only.

I	a	b	+	*	$
I_2	S_4	S_5	r_2		r_2
I_3	r_4	r_4	r_4	S_8	r_4

↓ Not LR(0) but SLR(1)

follow(E) = {+, $}

follow(T) = {+, $, a, b}

follow(F) = {+, *, $, a, b}

	a	b	+	*	$	Continue
I_7	r_3	r_3	r_3	S_8	r_3	
I_9	S_4	S_5	r_1		r_1	

] ⟶ LR(0) ✗ due to SR but conflict resolved in SLR(1)

I_2, I_3, I_7, I_9] Generates SR conflict but it get resolved in SLR(1)

So grammar is SLR(1) but not LR(0).

CLR(1) and LALR(1) Parsers

LALR(1) CLR(1)

Canonical collection of LR(1) items

LR(1) ⟶ LR(0) item + look ahead

$S \longrightarrow .aA$ [LR(0)]

LR(1): $S \longrightarrow .aA$, a/b look aheads.

Ex: $S \longrightarrow AA$
$A \longrightarrow aA \mid b$

$S' \longrightarrow .S$ ⟶ LR(0)
$S' \longrightarrow .S \mid \$$ LR(1)
$S \longrightarrow .AA, \$$
$A \longrightarrow .aA / .b, a/b$

Process → ① write augmented production LR(0) form

$S' \longrightarrow .S$

Since this is augmented, so make it LR(1) by adding look ahead $.

$S' \longrightarrow .S, \$$ [LR(1)]

② on $S' \longrightarrow .S$, (".") dot is before S so add all production with S on LHS

$S \longrightarrow .AA$

To find out lookahead take the every symbol existing just after the symbol or variable followed by dot to

$S \longrightarrow AA$ is due to $S' \longrightarrow .S, \$$ (2) so in (2) after .S is only $. So find first($) to get look ahead. first($) = $ itself. So LR(1) of $S \longrightarrow .AA$ is $\boxed{S \longrightarrow .AA, \$}$

Generalise

if $A \longrightarrow \alpha . B\beta, a/b$

then $B \longrightarrow .$ (anything), look ahead

look ahead = first(βa) / first(βb).

because after B there is $\beta(a/b)$.

So LR(1) for given grammar

$S \longrightarrow AA$

$A \longrightarrow aA \mid b$

LR(1)

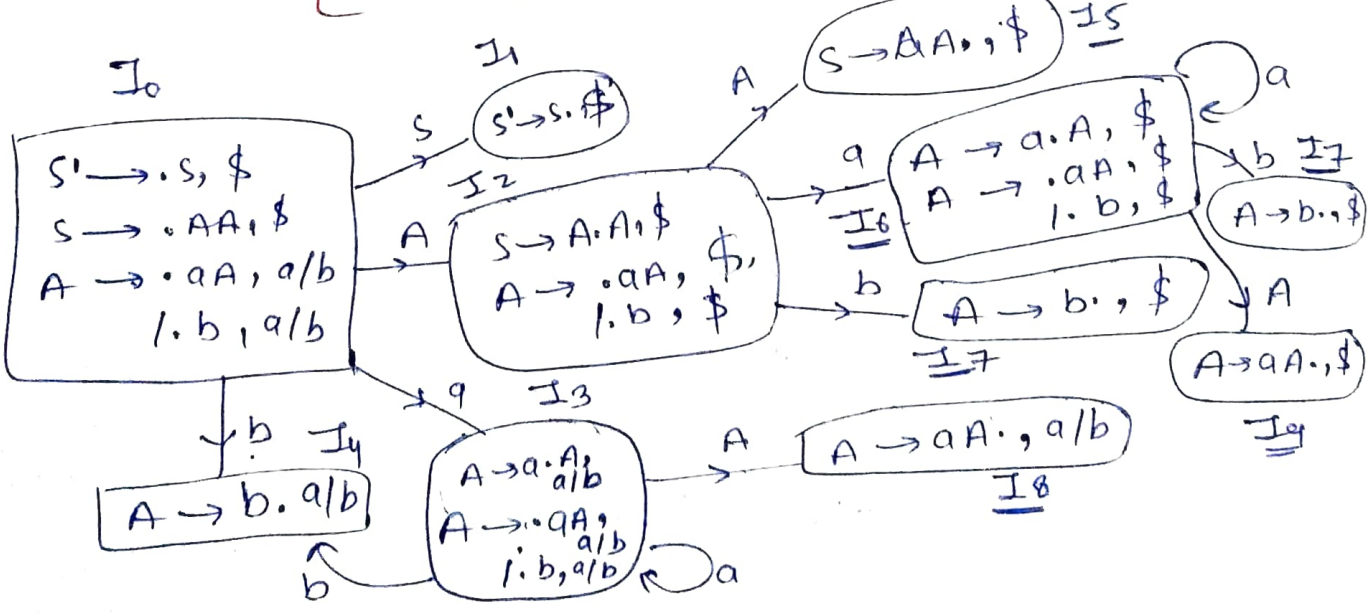
$I_0 \leftarrow$

$S' \longrightarrow .S, \$$

$S \longrightarrow .AA, \$$

$A \longrightarrow .aA$, first($A\$$) = a/b = $.aA, a/b$

$/. b$, first($A\$$) $/. b, a/b$



→ On applying transition look ahead might not changed but on applying closure look ahead will get changed.

No. of states in LR(1) canonical collection is more than LR(0) canonical collection for same grammar.

CLR(1) parsing table no. of states is equals to no. of states in LR(1) canonical collection. diagram.

In CLR(1) the reduced item will get placed in the look aheads.

In LR(0) we placing the reduced item in complete row but in SLR(0) we were placing it in follow(LHS) but in CLR(1) placing in only look aheads only. It reduce the no. of conflicts, but due to this Blank spaces and error detection capability is going to increased

① I_3, I_6 → both have same LR(0) items but different look aheads.

② I_4, I_7

③ I_8, I_9

CLR(1) Parsing table

	a	b	$	S	A
0	S_3	S_4		1	2
1			accept		
2	S_6	S_7			5
3	S_3	S_4			8
4	r_3	r_3			
5			r_1		
6	S_6	S_7			9
7	r_2	r_2	r_3		
8			r_2		
9					

Shift move and goto move are same as LR(0) table but reduce moves are different

$S \rightarrow AA$ ... r_1

$A \rightarrow aA$; r_2

$A \rightarrow b$; r_3

Table have very big size so we can reduce the size by merging the space with same LR(0) as

$[I_3, I_6] \rightarrow I_{36}$ $[I_4, I_7] \rightarrow I_{47}$, $[I_8, I_9] \rightarrow I_{89}$

no. of states in CLR(1) $\geq$ no of states in LR(0) or SLR(1)
No of states LR(0) = SLR(1)

↓

After merging some spaces it becomes LALR(1).

$$CLR(1) \geq LR(0) = SLR(1) = LALR(1)$$

↳ No. of states.

then LALR(1) parsing table look like

CLR(1) Reduced to LALR(1).

	a	b	$	S	A
0	S_{36}	S_{47}			2
1					
2	S_{36}	S_{47}			5
36	S_{36}	S_{47}			89
47	r_3	r_3			
5			r_1		
36	S_{36}	S_{47}		89	
47			r_3		
89	r_2	r_2			
89			r_2		

union

⇒

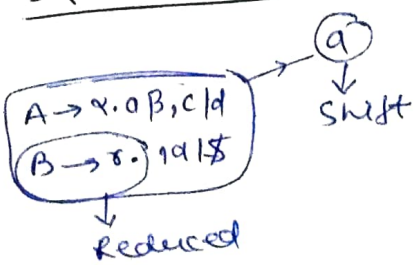
	a	b	$	S	A
0	S_{36}	S_{47}			2
1					
2	S_{36}	S_{47}			5
36	S_{36}	S_{47}			89
47	r_3	r_3	r_3		
5			r_1		
89	r_2	r_2	r_2		

union

Deleting duplicates after finding union.

LR(1) items

SR conflict → Dependson look ahead ← RR conflict



A → α. , a
B → α. , a

↳ Both are reduced and final but due to same look ahead both get placed in same column

If a grammar is not CLR(1) then it must not be LALR(1).

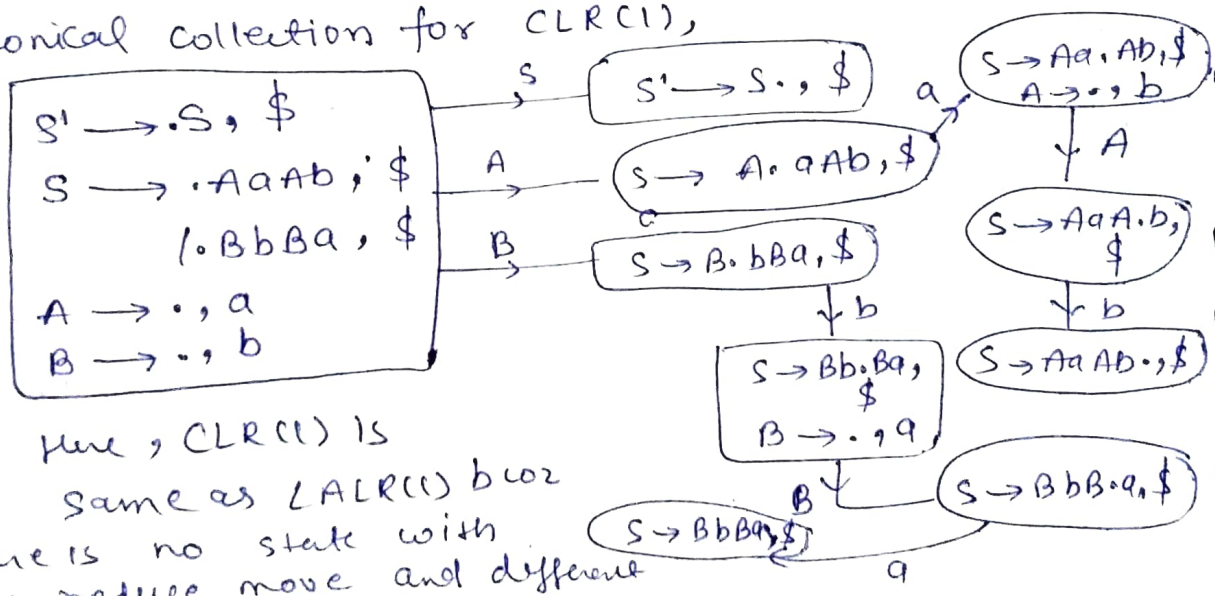
If a grammar is CLR(1) then it is not necessarily LALR(1)

If there is no SR conflict in CLR(1) then it must not be in LALR(1), but if there is no RR conflict in CRL(1) it is not necessary that there is no RR conflict in LALR(1).

Ex $S \rightarrow AaAb \mid BbBa$
$A \rightarrow \epsilon$
$B \rightarrow \epsilon$

① LL(1) ✓
② LR(0) ✗
③ SLR(1) ✗
④ CLR(1) ✓
⑤ CLR(2) ✓

Canonical collection for CLR(1),



→ Here, CLR(1) is same as LALR(1) because there is no state with same reduce move and different look aheads.

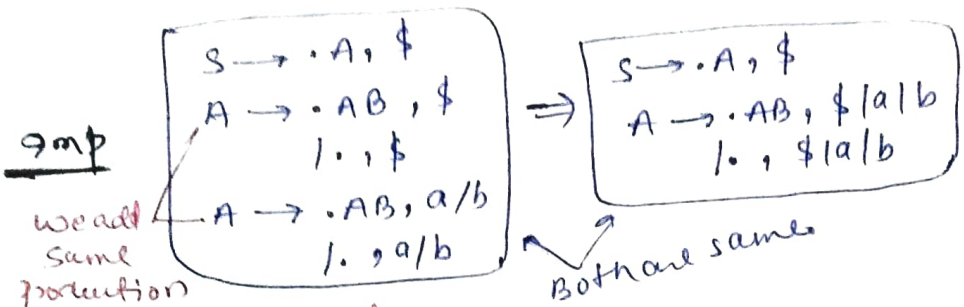
→ On terminal it is shift and on variable it is goto.

Example of CLR(1) and LALR(1) and comparison of all parsers.

Grammar
$S \rightarrow A$
$A \rightarrow ABE$
$B \rightarrow aB \mid b$

Closure (S → .A , $)

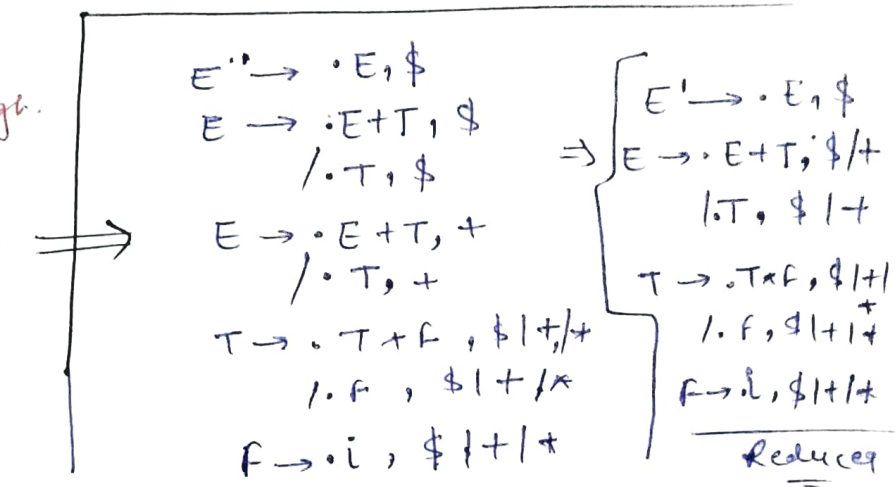
↓



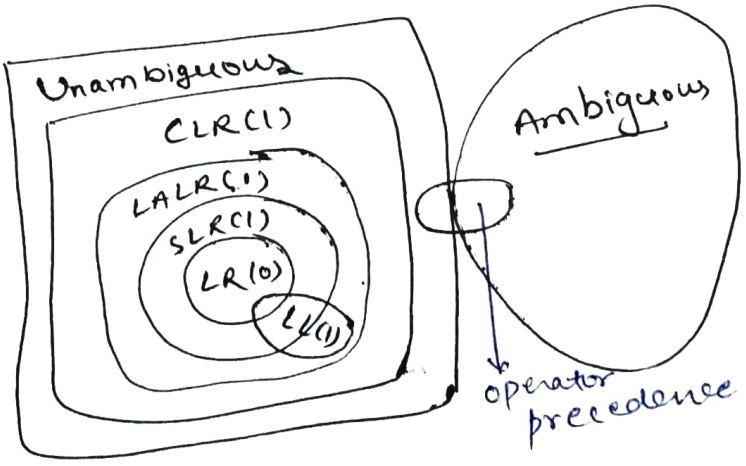
Imp

We add same production twice bcoz we get different look aheads, but in case of 2 R(0) only one production is enough.

② E → E+T | T
T → T*F | F
F → i



Power of Parsers



* If grammar is LL(1) then it must be LALR(1).

* In SLR(1) and LALR(1)
① Goto moves will always identical.
② Shift moves
③ Reduce entries in tables may be different.
④ Error entries in table may be different.
⑤ Both have same no. of states.

$S \rightarrow CC$

$C \rightarrow cC/d$

a) LL(1)

b) SLR(1) but not LL(1)

c) LALR(1) but not SLR(1)

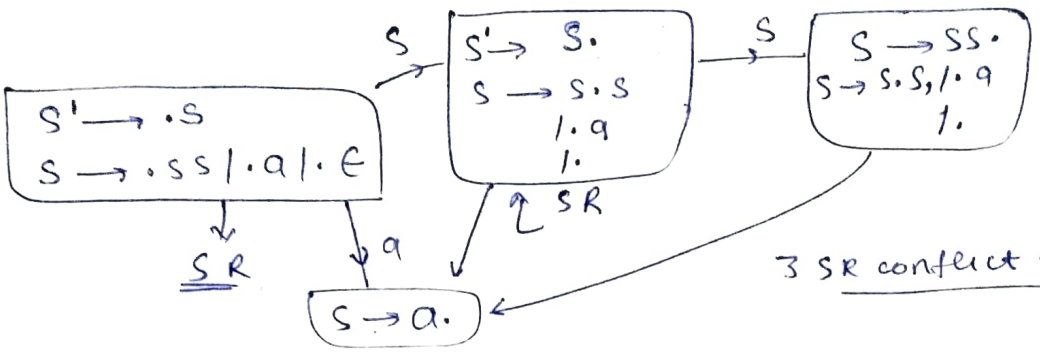
d) CLR(1) but not in LALR(1)

→ Grammar is LL(1), LR(0), SLR(1), CLR(1) and LALR(1) all of them.

find the number of SR and RR conflicts in dfa with LR(0) items.

$S \rightarrow SS/a/\epsilon$

Both (SR and RR)



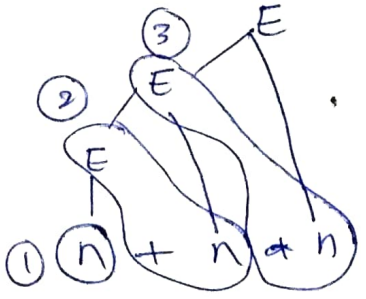
3 SR conflict + 1 RR conflict

→ Total 4 conflicts are generated

Q $E \rightarrow E+n / E*n / n.$

For the string $n+n*n$, the handles in right sequential form of reduction are-

Right sequent form: Bottom-up parsing.



Handles means the element that is being reduced.

→ firstly n is reduced so first handle is 'n'

then we reduced (E+n) to E → 'E+n' is second handle

then we reduced (E*n) to E → 'E*n' is another handle

Option a) $n_1, E+n, E+n+n$

b) $n_1, E+n, E+E*n$ ✗

c) $n_1, n+n, n+n+n$

d) $n_1, E+n, E*n$ ⟶ correct one

Q $S \rightarrow (S) \mid a$

SLR(1) : n_1, [CLR(1) or LR(1)] : n_2, LALR(1) : n_3

n_1, n_2, n_3 are no. of states then,

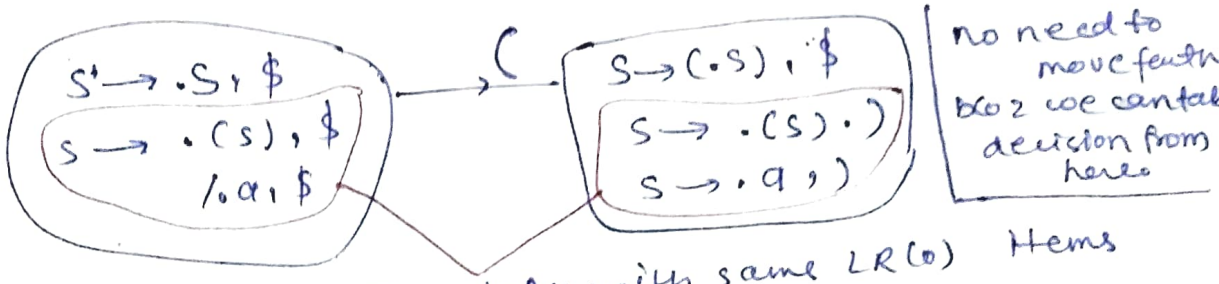
a) $n_1 < n_2 < n_3$

b) $n_1 = n_3 < n_2$ ✓

c) $n_1 = n_2 = n_3$

d) $n_1 \geq n_3 \geq n_2$

Soln



Two states with same LR(0) Items but different look aheads in LR(1). So they will reduce to combine in LALR(1) table so

Size of LR(1) table > size of LALR(1).

$n_2 > n_3$

and size of LALR(1) = size of SLR(1)

$n_3 = n_1$

$$n_1 = n_3 < n_2$$

→ Ambiguous grammar can't use in parsing procedure bcoz in canonical structure all the final states will get conflicts but if somehow we can remove those conflicts meaningfully then we can use that grammar for parsing.

YACC: Yet Another Compiler Compiler → O/P [LALR(1)]

↳ It resolve both SR and RR conflicts

→ SR conflict resove by shift move

→ RR conflict resolve by first reduce

Syntax direct translation (SDT)

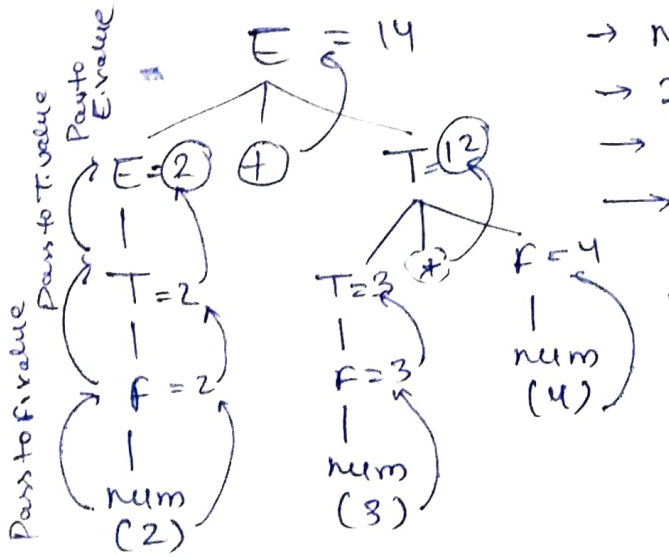
Syntax directed translations examples

Grammar + {Semantic rules} = SDT

SDT for evaluation of expression

$E \rightarrow E+T$	{ E.value = E.value + T.value }	(1)
$/ T$	{ E.value = T.value }	(2)
$T \rightarrow T*F$	{ T.value = T.value * F.value }	(3)
$/ F$	{ T.value = F.value }	(4)
$F \rightarrow num$	{ F.value = num.value }	(5)

Take a expression 2+3*4



→ num → token

→ 2, 3, 4 → lexical values

→ value → attribute

→ E, T, F → variables

→ In SDT every variable can have any number of attributes from 0 to n.

→ Bottom-up Parsing.

② $E \rightarrow E+T \mid T$ { Printf ("+"); } ①
{ } ②

$T \rightarrow T*F$ { Printf ("*"); } ③

$\mid F$ { } ④

$F \rightarrow num$ { Printf (num.lval); } ⑤

Production + Semantic rule = semantic action and we give no. to them

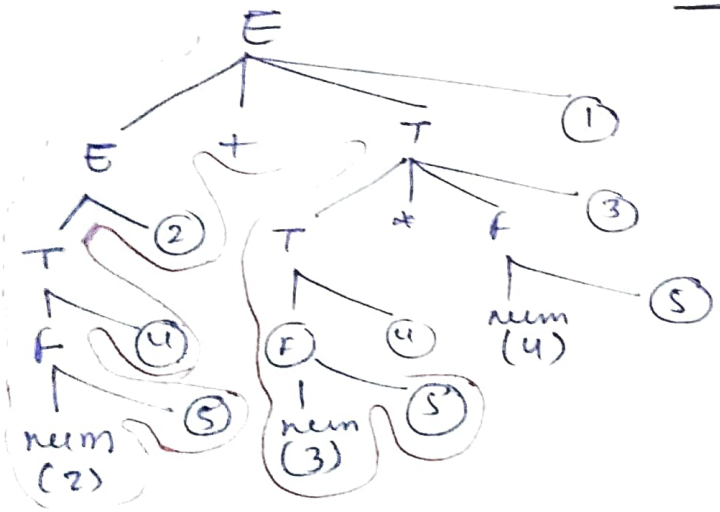
Above SDT is used to convert Infix to Postfix.

2 + 3 * 4 (Infix)

Postfix → 2 3 4 * +

→ Using top-down parser

we perform operation top → down and left to right and whenever we look semantic action we performed it.



o/p 2 3 4 * +

⑤ ④ ⑤ ④ ⑤ ③ ①

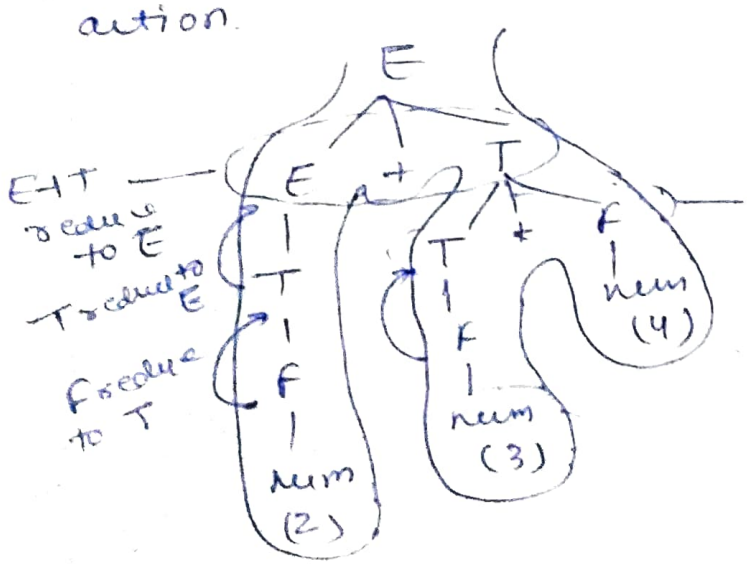
②

semantic action number and corresponding o/p

→ bottom-up parser

Mainly focused on reduction.

for reduction they take production and perform action.



o/p 2 3 4 * +

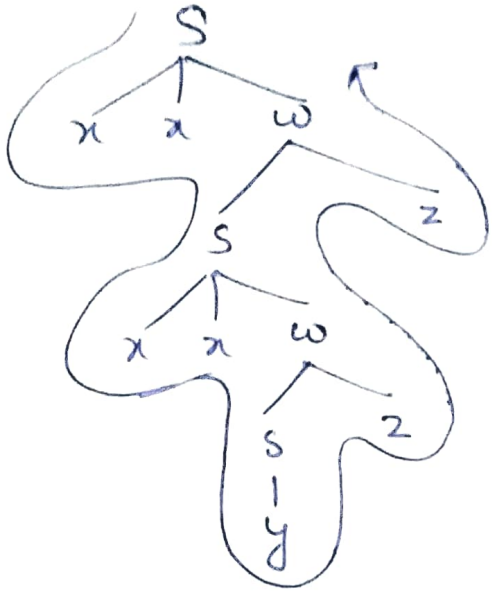
T * F reduce to T

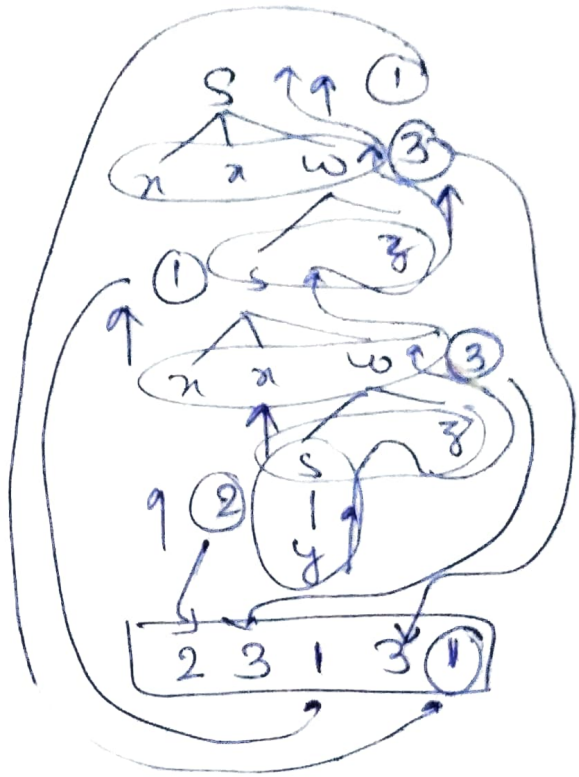
→ we get same output so SDT can be use for both top down and bottom up parsers.

(2) $S \rightarrow xxw$ { Printf(1); }
$\quad / y$ { Printf(2); }
$w \rightarrow Sz$ { Printf(3); }

String : xxxxyzz

→ bottom up parser (LR).





(4) $E \rightarrow E + T$
$\quad / T$
$T \rightarrow F - T$
$\quad / F$
$F \rightarrow 2$
$\quad / 4$

Evaluate : 4 − 2 − 4 * 2

Observation from Grammar

(1) ⊖ have more precedence than ⊕ bcoz it comes at lower level.

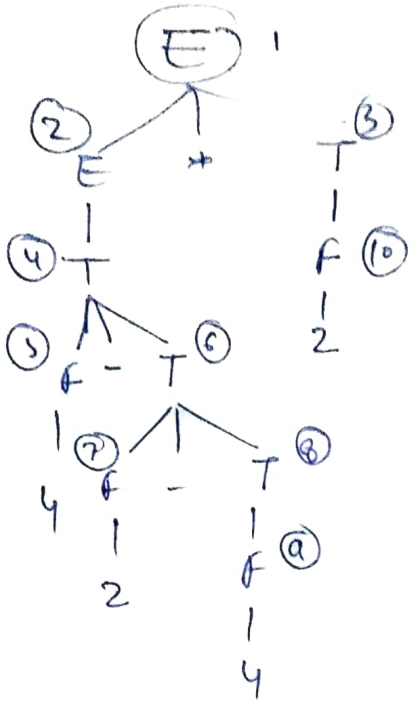
(2) ⊖ have Right to left predecence because Grammar production T is Right recursive.

So $((4 - (2-4)) * 2)$

-2

6

12 Ans.



No. of reductions = 10

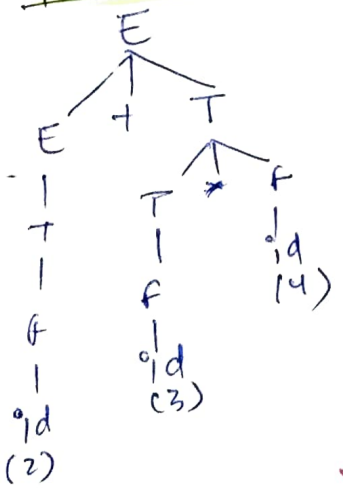
All the non-leafs are reduction

Note → Left associative means left to right say that grammar is left recursive. and same for right associative too.

Example of SDT

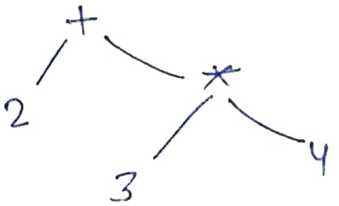
2 + 3 * 4

Concrete syntax tree or parse tree



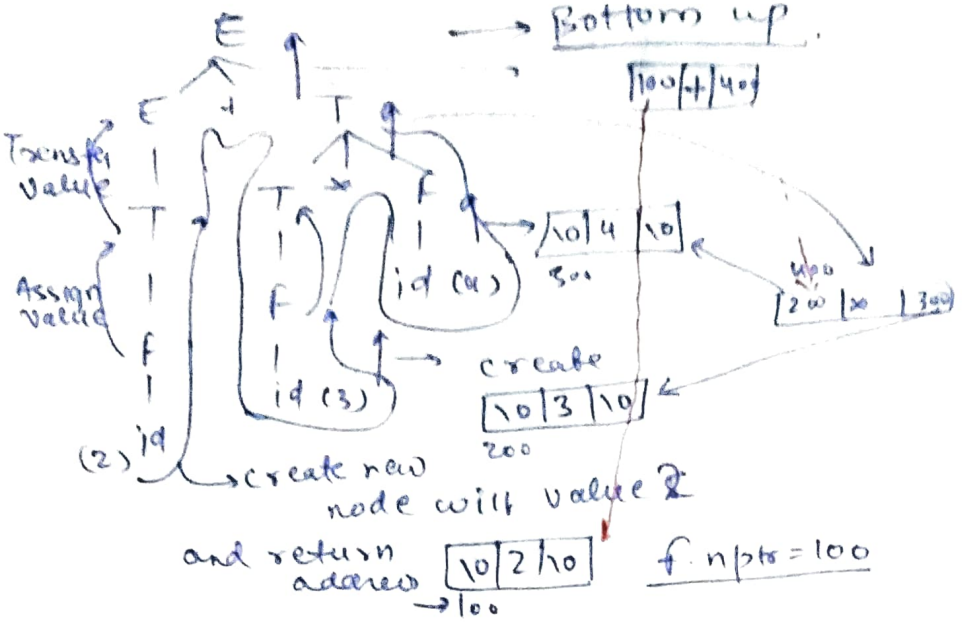
Abstract syntax tree or parse tree

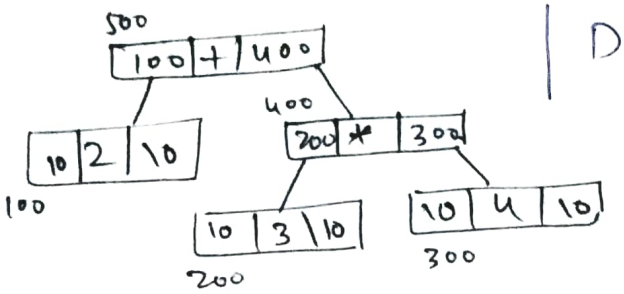
Expression tree



STD to built abstract syntax tree

$E \rightarrow E + T$	{ E.nptr = mknode (E1.nptr, '+', T.nptr); }
$\mid T$	{ E.nptr = T.nptr; }
$T \rightarrow T * f$	{ T.nptr = mknode (T1.nptr, *, f.nptr }
$\mid f$	{ T.nptr = f.nptr }
$f \rightarrow id$	{ F.nptr = mKnode (null, id.name, null); }





Data structure of abstract parse tree

STD for type checking

E → E1 + E2 { if((E1.type == E2.type) && (E1.type = int)) then E.type = int else error; }

| E1 == E2 { if ((E1.type == E2.type) && (E1.type = int / boolean) then E.type = boolean else error; }

| (E1) { E.type = E1.type; }

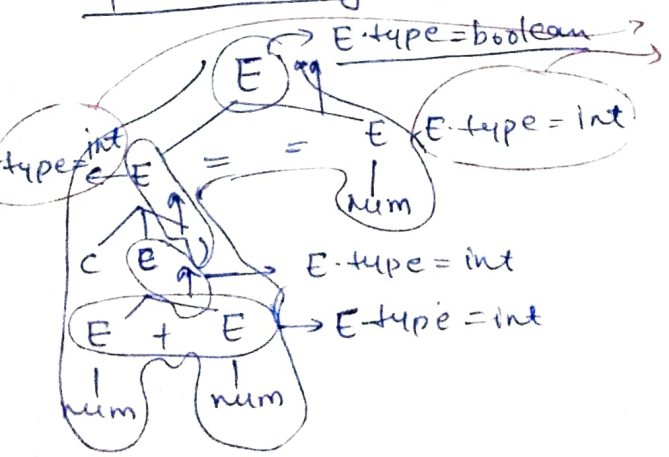
| num { E.type = int; }

| True { E.type = boolean; }

| false { E.type = boolean; }

⇒ Both the the E1 and E2 are same as E but just for separation btw LHS and RHS we make this.

operation string (2+3) == 8



So top most E will get E.type = boolean

	Count 1's	Count 0's
$N \rightarrow L$	{ N.count = L.count }	→ same
$L \rightarrow L_1 B$	{ L.count = L_1.count + B.count }	→ same
$/ B$	{ L.count = B.count }	→ same
$B \rightarrow 0$	{ B.count = 0 }	{ B.count = 1 }
$/ 1$	{ B.count = 1 }	{ B.count = 0 }

N → Number L → List of bits B → Bits

Convert binary to decimal using above grammar:-

MSB 1011 LSB

1 → 1x2+0 → 2x2+1 → 5x2+1 → 11

Calculate or count of bits before LSB, multiply it with 2 and add LSB.

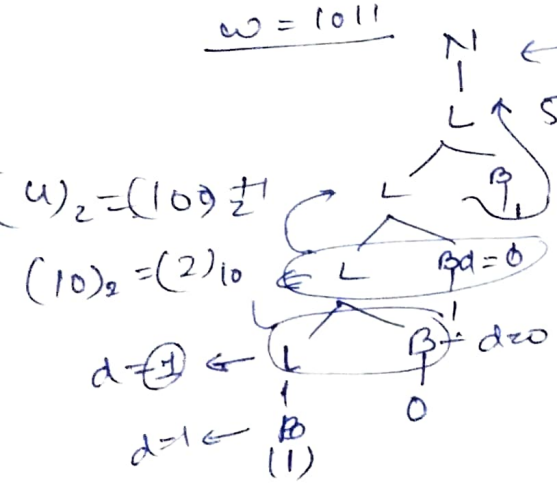
$N \rightarrow L$	{ N.dval = L.dval }
$L \rightarrow L_1 B$	{ L.dval = L_1.dval * 2 + B.dval }
$/ B$	{ L.dval = B.dval }
$B \rightarrow 0$	{ B.dval = 0 }
$/ 1$	{ B.dval = 1 }

L

1100 | 1

L_1 B

w = 1011



Binary no. contain decimal

$(11.01)_2 \rightarrow (3.25)_{10}$

$2^{-2} \times 1$

$2^{-1} \times 0$

$(11)_2 \rightarrow (3)_d$

$(.11)_2 \rightarrow \frac{3}{2^2} = 0.75$

$N \rightarrow L_1 . L_2$ { N.val = L_1.dval + (L_2.dval / $2^{L_2.count}$) }

$L \rightarrow L_1 B$ { L.count = L_1.count + B.count ; L.dval = L_1.dval + B.dval }

$/ B$ { List.count = B.count , L.dval = B.dval }

$B \rightarrow 0$ { B.count = 1 , B.dval = 0 }

$/ 1$ { B.count = 1 , B.dval = 1 }

Both L_1 and L_2 are same

SDT to generate 3-address code

S → id = E { gen (id.name = E.place) }

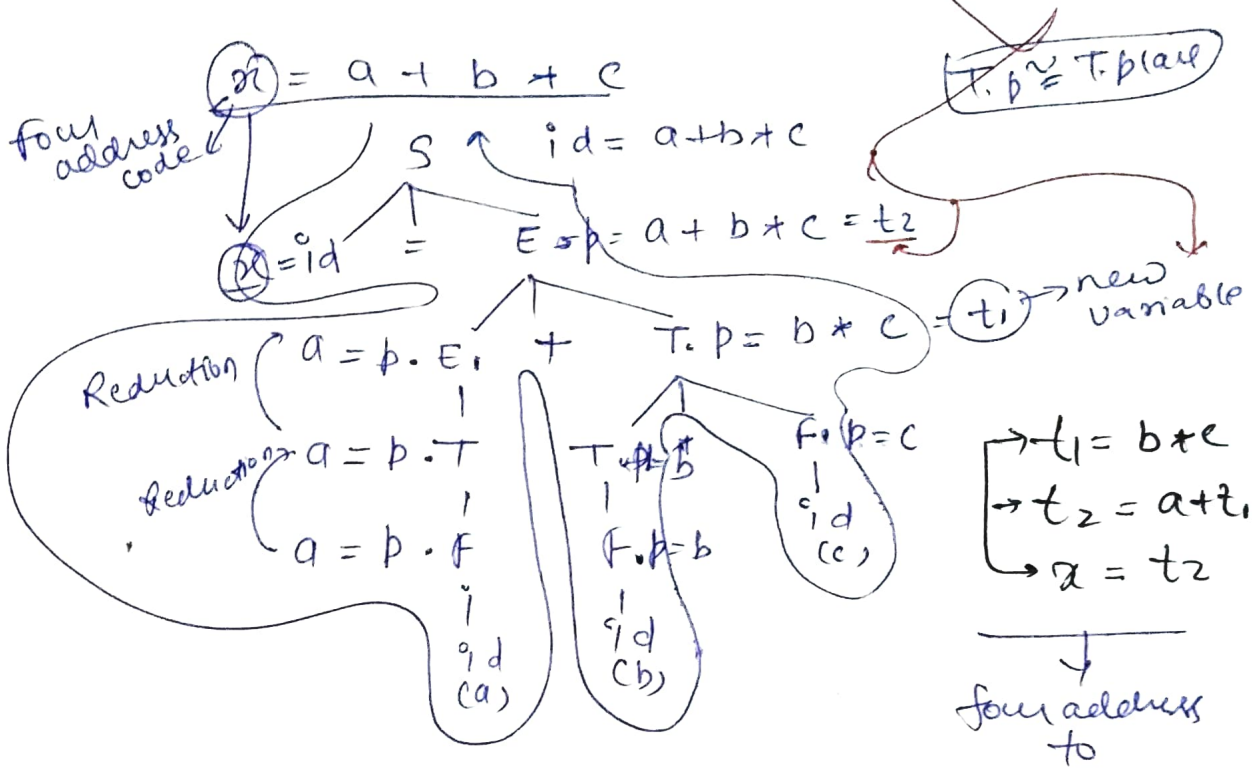
E → E1 + T { E.place = newtemp() ; gen (E.place = E.place + T.place) }

| T { E.place = T.place }

T → T1 * F { T.place = newtemp() ; gen (T.place = T1.place * F.place) }

| F { T.place = F.place }

F → id { F.place = id.name }



Classification of SDT and attributes

Synthesised attribute

Parent ← A → BCD → child

A.att = f (B.att , C.att , D.att). means parent is taking attribute from children.

Inherit Attribute

A → B C D

C.att = f (A.att , B.att , D.att) here 'C' can be replaced with B and D only means child in inhering attribute from parent.

S – attributed SDT

① Use only synthesized attributes

② Semantic actions are placed at right end of productions.

$A \rightarrow BCC \{ \}$

↓

also called as postfix SDT

③ Attributes are evaluated using bottom up parsing

L-attributed SDT

① Can have both kind of attribute but inherit only from parent or left siblings

Ex $A \rightarrow XYZ$

$\{Y.S = A.S, Y.S = X.S, Y.S \neq Z.S\}$

② Placed anywhere on RHS

$A \rightarrow B \{ \}$
$\quad / C \{ \} D$
$\quad / \{ \} E$

③ Traverse parse tree depth first, left to right.

Ex $A \rightarrow LM \{ \underbrace{L.i = A.i}_{\text{① Inherit}}, \underbrace{M.i = L.S}_{\text{② L attribute}}, \underbrace{A.S = M.S}_{\text{③ L attribute}} \}$

Not S

combiningly is it L attribute SDT

$A \rightarrow QR \{ \underbrace{R.i = A.i}_{④ L}, \underbrace{Q.i = R.i}_{⑤ \text{Not-L}}, \underbrace{A.S = Q.S}_{⑥ L} \}$

Due to ① it is not S attributed and due to ⑤ it is not L attributed. So it is neither L attributed or neither S attributed

SDT to store type information in symbol table

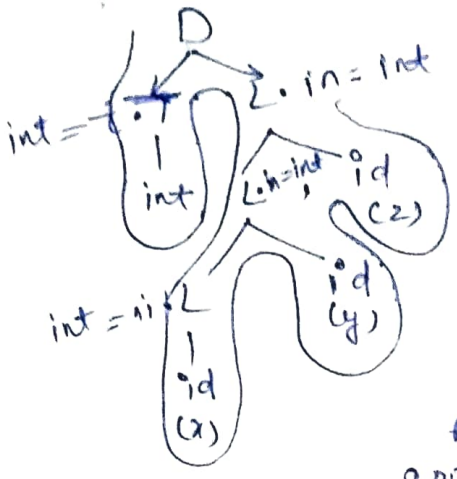
$D \rightarrow TL \{ L.in = T.type \}$

$T \rightarrow int \{ T.type = int \}$
$\quad / char \{ T.type = char \}$

$L \rightarrow L_1, id \{ L_1.in = L.in, addtype(id.name, L.in); \}$
$\quad / id \{ addtype(id.name, L.in) \}$

Above grammar is L-attributed grammar.

Ex → int x, y, z



(evaluate first visit)

Visit on each id

x	int
y	int
z	int

When you came across any inherited attribute evaluate first time when you visit it and across synthesized attribute evaluate last time when you visited.

All the L are inherited attribute so apply the action on first visit.

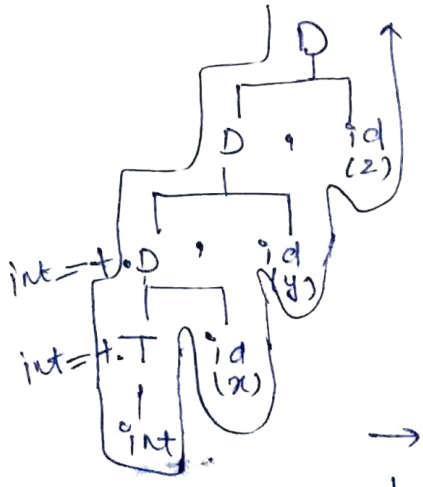
S → attributed SDT (Only synthesized attributes)

D → D_1 , id { addtype (id.name , D_1.type), D.type = D_1.type }
/ T, id { addtype (id.name , T.type), D.type = T.type }

T → int { T.type = int }
/ char { T.type = char }

Both D and D_1 are same

Ex → int x, y, z



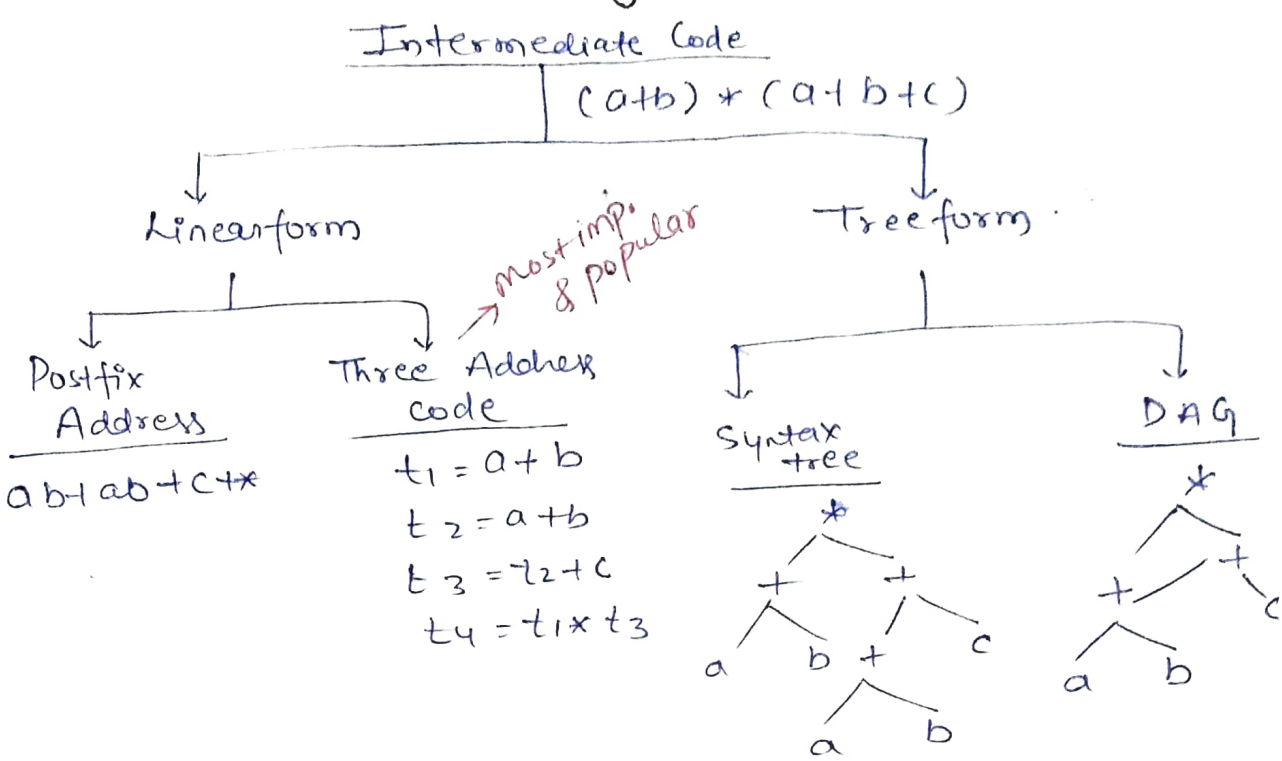
x	int
y	int
z	int

On each visit to id we add T.type and id.name in table.

→ In S-attribute we will do action on last visit to the attribute.

⇒ To convert a SDT from L attributed to S-attributed we need to change grammar and semantics action both.

Intermediate code generation



Syntax tree → Condensed and modified version of parse tree.

DAG (Directed Acyclic Graph) → It doesn't repeat anything and use all the things that is already generated.

To remember

Types of 3 address code -

1. $x = y \ op \ z$
2. $x = op \ z$ (Unary)
3. $x = y$
4. if x (relop) y goto L
5. goto L
6. $A[i] = x$
   $y = A[i]$
7. $x = *P$
   $y = \&y$

⇒ We convert Higher level code to three address code in order to generate the machine code.

Various representation of 3-address code.

Ex → (a+b) + (c+d) + (a+b+c)

	Quadruples				Triple				Indirect Triples		
		opr	op1	op2	res		opr	op1	op2		
$t_1 = a+b$	1)	+	a	b	t_1	1)	+	a	b	(i) (1)	→ Pointer to instruction
$t_2 = -t_1$	2)	−	t_1		t_2	2)	−	(1)		(ii) (2)	→ instruction to be executed
$t_3 = c+d$	3)	+	c	d	t_3	3)	+	c	d	(iii) (3)	
$t_4 = t_2 * t_3$	4)	*	t_2	t_3	t_4	4)	*	(2)	(3)	(iv) (4)	
$t_5 = a+b$	5)	+	a	b	t_5	5)	+	a	b	(v) (5)	
$t_6 = c+t_5$	6)	+	t_5	c	t_6	6)	+	(5)	c	(vi) (6)	
$t_7 = t_4+t_6$	7)	+	t_4	t_6	t_7	7)	+	(4)	(6)	(vii) (7)	
	adv. statements can be move around				adv. space is not wasted				adv. statement can't be moved		
	dis: too much of space is wasted				dis. statement can't be moved				dis. two memory access.		

Order can be changed bcoz all statements are independent

Instrn order can't be change bcoz statement are dependent

Back patching and conversations to 3 address code

if (a<b) then t=1
else t=0

] four address code but we can implement it using 3 address code.

(i): if a<b goto (i+3) — ① Blank
(i+1) : t=0
(i+2) : goto (i+4) — ② Blank
(i+3) : t≠1
(i+4) :

leaving the tables as empty and filling them later called as back patching.

Explaination → if a<b then goto instruction at (i+3) else (i+1) automatically get executed because it comes just next. Either (i) or (i+1) will get executed and after that pointer comes at (i+2) then exit the code.

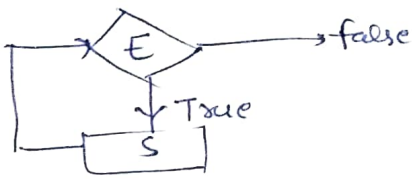
If a<b and c<d or e<f

100) if a<b goto 103
101) t1 = 0
102) goto 104
103) t1 = 1
104) if (c<d) goto 107
105) t2 = 0
106) goto 108
107) t2 = 1
108) if (e<f) goto 111
109) t3 = 0
110) goto 112
111) t3 = 1
112) t4 = t1 and t2
113) t5 = t4 or t3

3 address code using Back patching

while loop → If is not in 3 address code

while E do S



① L: if (E == 0) goto L1 (T → means condn is false)
S
goto L
L1:
] 3 address code

② L: if E goto L1] → If E is true goto execute L1 else
goto Last] → move to last

L1: S
goto L

Last: Code other than while loop in program.

while loop can be execute in 3 address code using various other methods too.

```
while (a<b) do
x = y+z
```

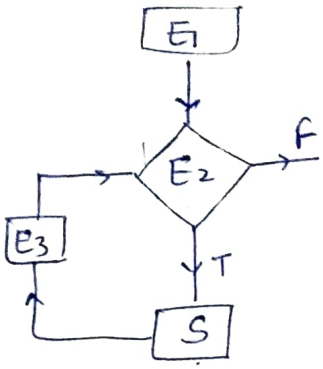
3-Add. code

It may fall in infite loop but our intention is just to understand the concept

L: if a<b goto L1
goto Last
L1: t = y+z
x = t
goto L

Last: Code other than while loop in programme.

for loop: for (E_1 ; E_2 ; E_3)
S



During execution of while loop copiler doesn't know we are executing while but compiler executes its prewritten 3-address code.

```
for (i=0; i<10; i++)
    a = b+c
```

```
    i = 0
L : if (i<10) goto L1
    goto Last
L1: t1 = b+c
    a = t1
    t = i+1
    i = t
    goto L
Last : Remaining code
```

Switch case:

```
Switch (i+j)
{
  case (i) : a = b+c;
             break;
  case (ii) : P = Q+R;
              break;
  default : x = y+z;
            break;
}
```

```
    t = i+j
    goto test
L1: t1 = b+c
    a = t1
    goto Last
L2 : t2 = Q+R
     P = t2
     goto Last
L3: t3 = y+z
    x = t3
    goto Last
test : if (t==1) goto L1
       if (t==2) goto L2
       goto L3
Last : Remain codes
```

2D array to 3-address code

A: 10 X 20 and we want to access

$x = A[y,z]$

$t_1 = y * 20$] → to compute position.

$t_2 = t_1 + z$

$t_3 = t_2 * (4)$] to compute base address of element

t_4 = base address of A

$x = t_4[t_3]$

This size (4 byte) is compiler dependent

Contn. next page

Explaination A[4][4]

	0	1	2	3
0	00	01	02	03
1	10	11	12	13
2	20	21	22	23
3	30	31	32	33

→ 2D array can be stored in m/m using two types

① Row major order

00 01 02 03 10 11 12 13 20 21 22 23 30 31 32 33

② Column major order

00 10 20 30 01 11 21 31 02 12 22 32 03 13 23 33

$x = A[2,3]$ (i, j) colum = 3 row = 2 but in memory the location somewhere else. So the code that converts Hight level code position to m/m location is 3-address code. we consider row major order.

$A[2,3] = (2 * 4 + 3) + 1 \rightarrow$ 12 position

(cross 11 element to get element at)

$A[i,j] = (i \times (\text{no. of col.}) + j) + 1$

→ Array of any dimension will get store in one dimension in memory.

Gate: 2007

In a simplified computer - - - - - - - - - - for this basic

① ADD R_1, R_2
$R_1 + R_2 \rightarrow R_2$

② ADD R_1, m
$R_1 + m \rightarrow m$

MOV	a, R_1
ADD	b, R_1
MOV	c, R_2
ADD	d, R_2
SUB	e, R_2
SUB	R_1, R_2
MOV	R_2, m

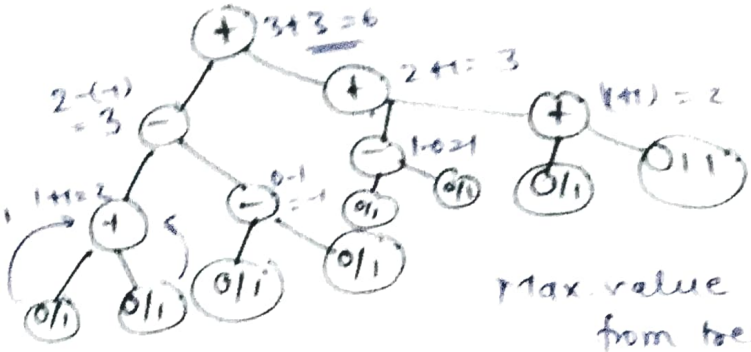
⇒ Total 3 move operations

Question Gate PYQ Book में 2 लगी है.

Gate-2004

Consider the grammar rule $E \rightarrow E_1 - E_2$ - - -

Given in videos must do on your own

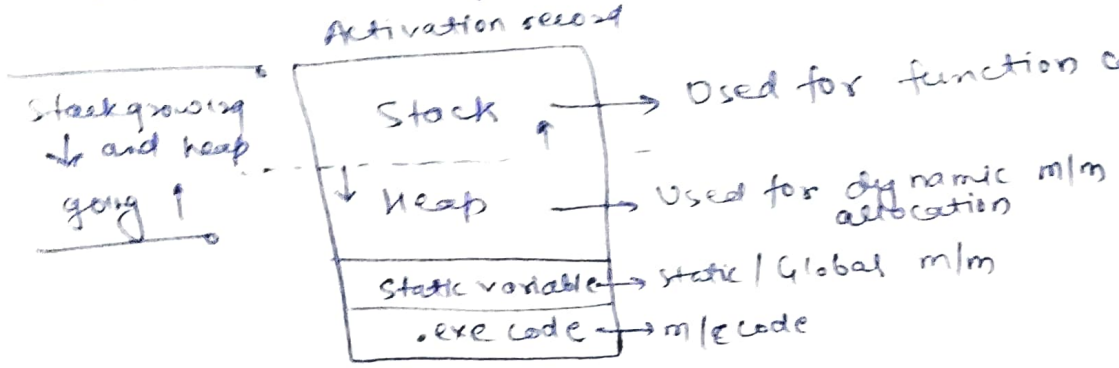


→ Remember the trick to solve this question.

Max value that can be obtain from tree = 6

Runtime - Environments

RTE means on the time of loading of programme to OS what kind of support you needed from OS.



Storage Allocation Strategies

1) Static : ① Allocation done at compile time.
② Binding do not change at run time.
③ One activation record for procedure.

V.table can be alive through out the program

Disadvantage : ① Recursion is not supported.
② Size of data object must be know to compile time.
③ Data structure can't be created / supported dynamically.

2) Stack : → whenever a new activation begin, activation record pushed on stack, and when activation ended it popped off.

→ Local variable are bound to fresh storage.

→ Dis : local variable can't return once the activation record popped off.

3) Heap : - Allocation and dellocation can be in any order.

Dis → Heap management is overhead. We can deal with overhead using first fit in OS.

Summary → Activation / function call can have —

i) Permanent lifetime in case of static allocation.

(ii) Nested lifetime in case of stack allocation.

(iii) Arbitrary lifetime in case of heap allocation.

Code optimization / Code generation (Not for Gate Exam)

→ Read it 'must' for the sake of knowledge

Introduction

Optimization (use to reduce no. of lines in code)

Machine Independent

1) loop optimization
   a) code motion or frequency reduction
   b) loop unrolling
   c) loop jamming
2) folding
   - constant propagation
3) Redendancy elimination
4) strength Reduction

Machine dependent

1) Register allocation
2) use of addressing modes
3) Peephole optimization
   a) Redundant load / store
   b) flow of control optimization
   c) strength reduction
   d) use of m/c idioms.

Loop optimisation and basic blocks

① To apply loop optimization, we must first detect the loops.

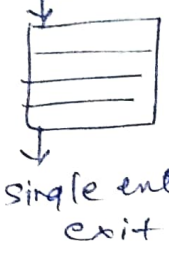
② For detecting loops we use control flow analysis (CFA) using program flow graph (PFG).

③ To find PFG we need to find basic blocks.

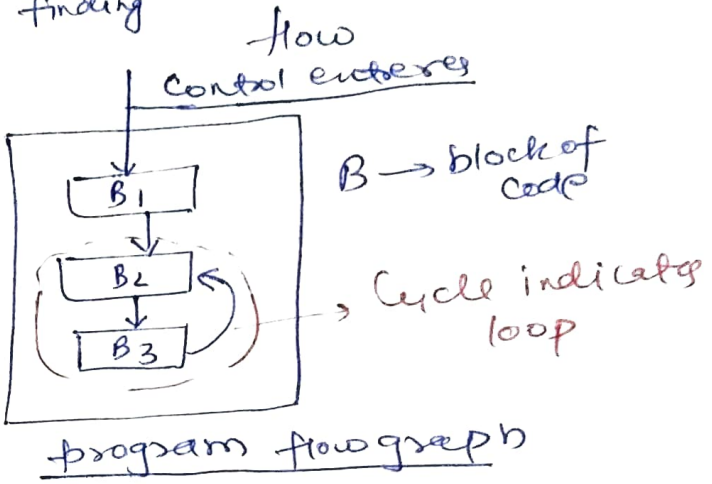
A block is sequence of 3 address statements where control enters at the beggining and leaves only at the end without any jump or halts.

HLL have (for loop, while loop) but when its concern to 3 address code then there is no visibility of loops and then it comes to process under code optimisation.

Basic Blocks → CFA (PFG) → loops finding → optimization applied



Single entry and exit point



program flowgraph

Algorithm to find the basic block.

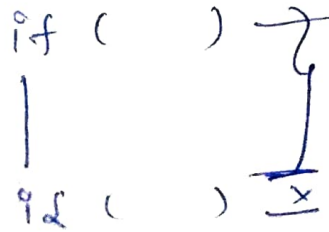
In order to find basic block, we need to find the leaders in program. Then a basic block will start from one leader to the next leader but not including next leader.

Identifying leaders in the block

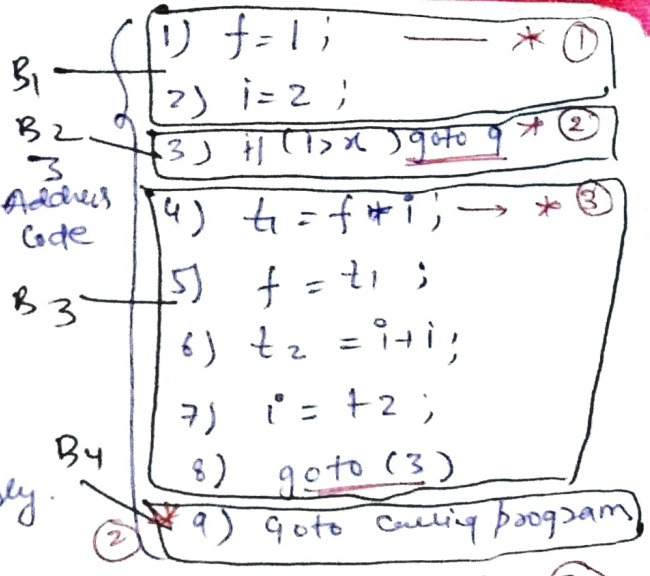
1) I statement is a leader
2) Statement that is target of conditional or unconditional statement is a leader.
3) Statement that follows immediately a conditional or unconditional goto statement is a leader.

(3) conditions

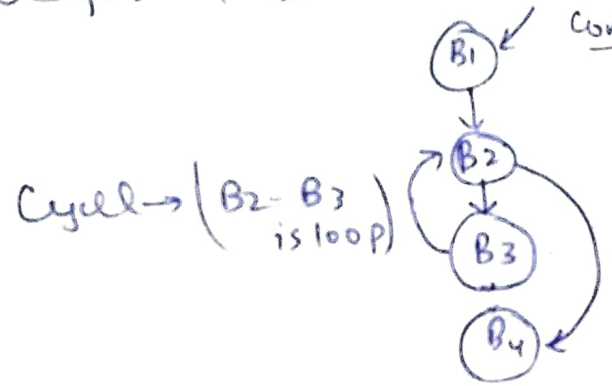
→ Block is a code between two leaders includes first leader statement only



Ex
```
fact(n)
{
  int f = 1
  for(i=2; i<=n; i++)
      f = f*i;
  return f;
}
```



We have 3 conditions to find leader as discussed previously.
So,
* ③ → means statement is leader due to condition ③
We get 4 basic blocks due to four leader.



→ This way we can identify the loop using (PFG)

Types of loop optimization

Loops can be optimize in two ways -
① either remove the loops
② reduce the lines in loop

① **Frequency Reduction**: Step after finding loops.
Move the code from high frequency region to low frequency region called code motion

ex-
```
i=0
while(i<5000)
{  A = sin(x) * i;
   i++;
}
```
⇒
```
t = sinx/cosx
while(i<5000)
   A = t * i
```

→ sinx/cosx eventime will have same value so calculate it once and save the calculation time

loop unrolling: → Reduce the no. of times we enters in loop & reduce the no. of comparison.

```
while(i<10)                while(i<10)
{ x[i]=0          ⇒        { x[i]=0
  i++                        i++
}                            x[i]=0
                             i++;
                           }
```

work on array that have size divisible by 2.

loop jamming:- Combine two loops together or reduce no. of loops.

```
for(i=0; i<10; i++)          for(i=0; i<10; i++)
  for(j=0; j<10; j++)        { for(j=0; j<10; j++)
    x[i,j]=0                   {
                                 x[i,j]=0
for(i=0; i<10; i++)              x[i,i]=0
  x[i,i]=0                     }
                             }
```

Machine Independent Optimization

folding: Replacing an expression that can be computed at compile time by its values.

Ex: $2+3+C+B = \underline{5+C+B}$.

Redundancy elimination: (DAG) [Use already evaluate values]

$A = B+C$

$D = 2+B+3+C$

$D = 2+3+A = \underline{5+A}$

Strength Reduction: Replace a costly operation by cheaper 1.

→ * and / are costly operations.

Ex $A = A*2$

$A = A<<1$

Algebric simplification

$A = A + 0$ } eliminate such statement bcoz they doesn't effect anything.

$x = x * 1$

Machine dependent optimization

① Register allocation
- local allocation
- global allocation

] Depends on how many register you have.

② Use of addressing modes

③ Peephole Optimisation

(a) Redundant load and store elimination

$x = y + z$

```
mov y, R0
add z, R0
mov R0, x
```

$a = b + c$
$d = a + e$

```
mov b, R0
add c, R0
mov R0, a
mov a, R0
add e, R0
mov R0, d
```

→ Load and store problem. So we can remove it.

(b) Flow of control optimization

- Avoid jumps on jumps
- eliminate dead code

L_1: jump L_4

L_2: jump L_3

L_3: jump L_4

```
#define x 0
if(x)
{
    ...
}
```

Dead code

→ bcoz x is never going to be 1.

never get execute

(c) use of machine idioms

$i = i + 1$

```
mov R0, i
add R0, 1
mov i, R0
```

⇒ inc i

Points to write / add additionally

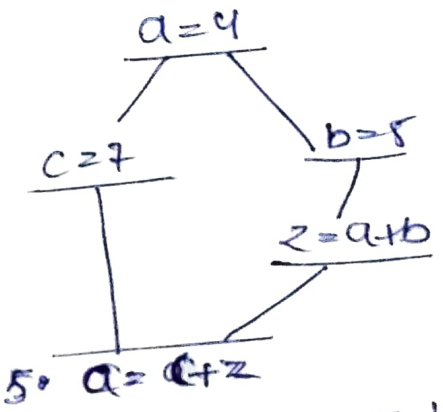
Liveness Analysis

→ Also called as live variable analysis.
→ Data-flow analysis to calculate the variables that are live at each point in a program

Adv. → Used in dead code elimination
Used for register allocation

① b = 3
↓ → b is live
c = 5
↓ → b & c are live here.
a = b + c

②



a → killed at 5 but we redifed at

Compiler Design

HLL → Preprocessor → Compiler → Assembler → Loader/Linker

Pure HLL — Assembly Language — Machine Code

HLL → (Character stream) Lexical Analysis → (tokens stream) Syntax Analyzer → ICG → TCG

Semantic Analysis

3 address code → CO (optional)

HLL → ICG : frontend

CO → TCG : Backend (System dependent)

Symbol table collect info. in analysis phase and use in synthesis phase.

Line of usage → Lines of code in which variable is used.

—— declaration → info. of line in which variable is declared.

Operations on symbol table depends on structure.

① Non block structured : Single instance of variable

② Block structured : multiple instance of variable

→ Complete string assumes as a single token.

Grammar → Left Recursive or right recursive

There is no direct method to verify ambiguity of grammar other than parse tree.

Ambiguity problem is undecidable. | Power is always right recursive.

Error recovery operations in lexical analysis: ① Delete ② Insert ③ Replace ④ Transpose

Ambiguity is generally due to —

① Associativity → Recursion ② Precedence. → level of operators.

Remove left recursion b/o2 Top-down parser doesn't support.

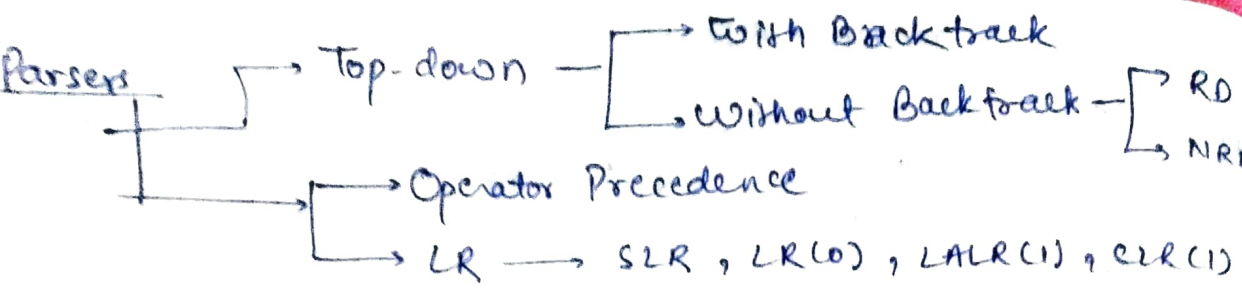
$A \rightarrow A\alpha \mid \beta$ then,

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Non-deterministic grammar generates common prefix problem, to remove it we use left-factoring method.

Even after left-factoring we can't say ambiguity is removed.



RD → Recursive descent , NRD → Non-Recursive Descent

TDP without backtracking can be done by removing left recursion & non-determinism.

TDP → Left most derivation

Bottom-up → right most derivation in reverse

LL(1) → no look ahead

L → left to right

L → Leftmost derivation

First() → It can contain ∈.

follow() → It doesn't contain ∈ but contain $ in follow() of starting symbol.

For LL(1) grammar must not be left recursive.

A → ∈ kind of productions kept in follow() of variable and other in first().

It can't contains multiple entries in a single column.

→ No ambiguous grammar is LL(1).

RDP:- We make recursive fn for every variable.

To solve recursive fn we use Stack and concept of activation records.

Operator precedence grammar:

In this grammar, no two operands come together.

We need an operator precedence table.

No relation among id & id or $ & $.

→ It takes $O(n^2)$ size. So overhead

To avoid we use operator function tables.

→ If graph has no cycle then construct fn tables.

It takes $O(n)$ size.

But disadvantage, it converts blank entries to valid entries.

→ Low error detecting capabilities but we use it anyhow.

→ Right recursive → right associative

→ Left recursive → left associative

→ Some times we can convert normal grammar to operator precedence grammar.

→ Operator precedence grammar can't handle (-)ve sign and it is for small grammar only.

LR(0) & SLR(1) • simple LR.

In LR(0) make the grammar augmented first then perform closure and goto operations on grammar to for a canonical collections of LR(0) items.

finally form a LR(0) passing table will all states and Action and Goto parts

→ In follow():

If follow(X) = ∈ or Null then add follow(V), where V is variable present as LHS of production in which we are searching for follow of (V).

→ In LL(1) first(S), S → any symbol/variable doesn't contain any same terminal twice

→ It should not be left-recursive (LL(1)).

for LR(0), there should be no SR/RR conflicts

→ If grammar is LR(0) then it must be SLR(1) otherwise it can be SLR(1) only.

→ In SLR verify that next traced symbol should present inside the follow (LHS) of previous reduced production.

→ If augment state or accepting state have conflict then we didn't accept the conflict

→ In CLR(1) keep the look aheads with the state too and keep the reduced move in table corresponds to look aheads.

→ In LALR(1) we merge the same entries of table together.

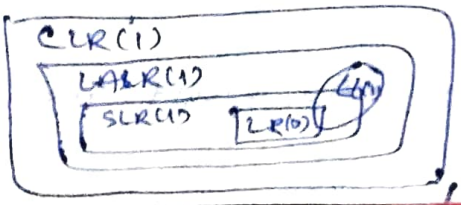
→ If a state has two reduce moves then it will work here if both have different look aheads.

If not CLR(1) then must not be LALR(1). If it is CLR(1) then it is not necessarily LALR(1).

No. of states:

CLR ≥ LR(0) = SLR(1) = LALR(1)

CLR(1) is most powerful among all.



→ SLR(1) detects the error earlier than LR(0).

→ In SR/RR conflict if both occur on same terminal or variable then neither SLR(1) nor LR(0). If both on different then it will be SLR(1) not LR(0)

→ If I_5 is reduced as r_1 and have production as $X \to y$ then we kept r_1 in follow(x). It avoids the conflicts a lot of times because in LR(0) we kept reduced moves blindly. It increases the chance of conflicts.

→ In CLR(1), it arises problem if there is a conflict at some state and both reduction moves have same look ahead.

In CLR(1): we get conflicts if.

$A \to \alpha . a / G$	a
$B \to r . / a, \$$	

Conflict SR on (a)

$A \to \alpha ., a$
$B \to \alpha ., a$

RR conflict on a

~~But try to remove this~~

→ In range from LR(0) → SLR(1) we try to reduce to reduce conflict and increase in error and blank space detecting capabilities.

In SLR(1) & LALR(1)

1. Goto and shift moves are always identicals
2. Reduce moves and error entries might get changed
3. Both have same no. of states.

If $S \to SS / a / \epsilon$ then I_0 will contains a SR conflict as

$S' \to .S$	S
$S \to .SS / .a / .\epsilon$	a

$S \to .\epsilon$ creates a reduced move

There is no shift move on $A \to .\epsilon$

→ YACC → O/p LALR(1). It resolves both SR & RR conflicts. SR resolved by shift move & RR resolved by first reduce move.

Syntax Direct Translation: Grammar + Semantic Rules.

→ Same grammar can behave different according to SDT

→ SDT gives same output on top down and bottom up both type of parsing.

→ Symbol that occur at lower level has high precedence.

→ Right recursive grammar → Right associativity of symbol and same for left recursion.

→ No. of reductions = no. of non-leaf in a tree.

→ SDT → Concrete Syntax Tree (Parse Tree)
→ Abstract Syntax Tree (Expression tree)

Types of attributes in SDT

Synthesise: $A \to BCD$: A.attr. = f(B.att, C.att, D.att).
Parent takes att. from their child.

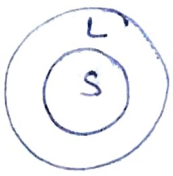
Inherit: $A \to BCD$: C.att = f(A.attr, B.att, D.att)
Child can take attribute from parents and any of its seiblings.

S attributed SDT

① Only Synthesized attr.

② Semantics can placed at last to
$A \to BC\{\}$

③ bottom up pasing



L-attributed SDT

① Both but inherit only from left & also synthesize.

② semantics can be placed anywhere
$A \to \{\}BC$, $A \to B\{\}C$,
$A \to BC\{\}$

⇒ Top-down, left → right

ICG: Types of 3 address Code

$x = y \text{ op } z$, $x = \text{op } z$ (unary) , $x = y$, if x (relop) y goto L,
goto L , $A[i] = x$ or $y = A[i]$, $x = \&p$, $y = *x$.

IC → Linear form → Postfix Addl. / 3 Addl. Code
→ Tree form → Syntax tree / DAG

Storage Allocation Strategies

① **Static**: Compile time allocation, no change at run time in binding, one activation record per procedure.

→ No recursion, size should be known at compile time, no dynamic data structure.

② **Stack**: Support recursion, local variable are fresh, local variable can't retain after popped off.

③ **Heap**: No order of allocation & deallocation, heap management is overhead, only support first fit in OS.

Activation record / fn call: Permanent lifetime in static, nested lifetime in stack, arbitrary in case of heap.

Code optimization or code generation

① **Machine Independent**: ① Loop Optimization [frequency reduction, loop unrolling, loop jamming], folding [Constant propagation], redundancy elimination, Strength reduction. $X=4, Y=2$ $Z=X+Y$, $T=Z+X+Y$, $T=\frac{Z+1}{2}$, algebraic simplification

② **Machine Dependent**: Register allocation, peephole optimization, [Redundant load/store, flow of control, use of m/c idioms].

Liveness Analysis / Constant propagation:

→ Lexical Analysis → it uses regular expression and finite automata

→ Type checking done at semantic analysis.

→ In L attribute SDT if all semantics rule are at end then we can use bottom up parsing.

→ Keywords of language are recognized during lexical phase

→ Code optimization (DAG), Abelian group (PDA), code generation (syntax tree).

→ $P \to \epsilon$ is also not an operator grammars

→ Control flow graph / flow graph both are same

→ character stream is input to lexical and tokens stream is input to syntactic phase.

→ CFG can be used to specify both lexical and syntax rules

→ In static single assignment form we use different variable register of same name at the time of assignment

→ In SR parsing stack contains only a set of viable prefix. Handle is actually on the top of the stack.
Viable prefix is prefix to handle so can never extend to right of handle (top of stack).

→ Lexical: DFA minimization, Parsing: Production tree, Register Allocation: Graph Coloring, exp. evaluation: postorder

→ SLR is very easy to implement.

(31)

→ Register spill: If all registers are full then transfer the content from register to memory.

→ In single-static we can't use same variable on LHS in expressions.

→ In CFG each variable (not registers) declare in separate state / block.

→ 3-address Code generation is a part of frontend. So no relation with backend. It is system independent.

→ In Grammar $S \to Sa \mid a$ [$S \to a$ is not a unit prodn]
but $S \to A$; $A \to a$ [$S \to A$ is unit production.]

→ Goto() always occur on non-terminals.

→ Two states in LALR gets merged if they have different look-aheads but same set of productions.

→ Some code-optimizations carried out on the intermediate code because it enhances the portability of compiler for other target processes.

→ LR(1) means CLR. LALR(1) will have SR conflict if only if CLR(1) have S-R conflicts

→ Every regular set has atleast one grammar which is unambiguous and LR(1).

→ By doing some modification in grammar, its language doesn't get changed.

→ Yacc gives o/p as LALR. It identified S-R conflict and resolve the conflict in favour of shift over reduce.

→ Handle is a substring that present in the RHS of substring. $S \rightarrow aB \mid a$, $B \rightarrow Bb \mid \epsilon$ [Only in Bottom up parsing]

String = abba
↳ a is handle in the stack

→ Viable Prefix : In shift/reduce parsing.
If string is not appearing on the stack then it will not be viable prefix.

Ex $S \rightarrow CC$; $C \rightarrow ac \mid b$

Str1 = ab

Stack	Prefix	
$	ab $	shift a
$a	b $	shift b
$ab	$	

$ab → viable

Str2 = bb

Stack	Prefix	
$	bb	shift b
$ b	b $	Reduce (→b)
$ C	b $	shift b
$ Cb	$	

$Cb → not bb

→ Global Variables are not present in the activation record of a procedure (C Acts / Control both linkage proc

→ Jumps are not allowed in the middle of basic block

→ DAG can be used to eliminate common sub expressions

→ White spaces removed in lexical phase, so not tokens

→ Leaf node of a tree is synthesized attributes.

→ In $B \rightarrow Bb \mid \epsilon$, first(B) = {b, ε}

→ In constant folding we did run time computations at compile time [Part of common subexpression]

→ In Chomsky Normal formal no. of nodes in parse tree for string of length $n = 2n-1$

→ In merging CLR → LALR, it might introduce R/R conflicts

→ In a string of length 'n' — no of subsequence (2^n), no. of prefix / suffix ($n+1$), [Including ∈] — no of proper prefix ($n-1$). [Excluding ∈ and string]

→ Every regular lang. doesn't follow LL(1).

→ CFL is closed under regular intersection.

→ Regular languages are closed under difference ($L_1 - L_2$) ($L_1 \cap \bar{L}_2$), reversal (DFA(L) $\xrightarrow{\text{reverse}}$ DFA(LR)), homomorphism, inverse homomorphism, quotient operation, substitution & infinite union.

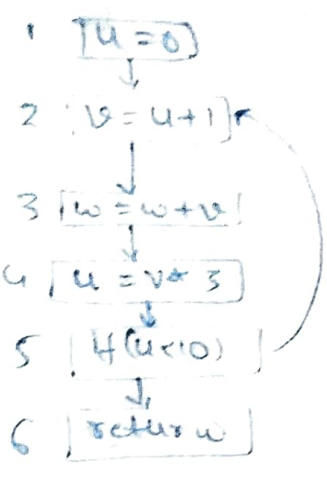
Rule 1 → $X = X + 3$; X is alive here.
$X = Y + 1$; X is dead here only Y is alive.

Rule 2 → In a line if variable is present at RHS, it is alive present at LHS follow Rule 1.

→ If you can reach 'nth' Instn in which variable without any definition then variable is alive.

	X	Y	Z	U	V	W
1	D	D	D	D	D	D
2	D	D	D	D	L	D
3	D	D	L	D	L	D
4	L	D	L	D	D	D
5	L	L	L	D	D	D
6	D	L	L	D	D	L
7	D	L	D	L	D	L
8	D	D	D	L	L	D

1. V = 1
2. Z = V + 1
3. X = Z * V
4. Y = X * 2
5. W = X + Z * Y
6. U = Z + 2
7. V = U + Y + W
8. return (V * U)



	u	v	w
1	D	D	L
2	L	D	L
3	D	L	L
4	D	L	L
5	L	D	L
6	D	D	L

→ flow of control remain same in both SSA and non-SSA form.

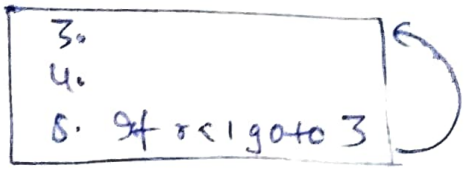
→ Let 'L' be a CFL. If every CFG with language $L = L(G)$ is ambiguous then L is said to be inherently ambiguous language.

→ Semantic errors occur at compile time.

→ CFG (parser) used in syntactic analysis. So it can capture matching curly braces, nested parantheses and syntax of if-else, but equality of datatype in RHS & LHS is work of semantic analysis.

→ RG (Lexical analysis): DFA, CFG (Syntax analysis) CSG (Semantic analysis).

→ We can move to same block at if as,



→ Also consider Entry and exit node inside CFG.

→ Automatic garbage collection is not essential to implement recursion.

→ The identification of common sub-expression and replacement of run-time computation by compile time computation is constant folding.

→ In DAG (a+a) can be represented as